data-science-ml: the Data Science and Machine Learning program — user guide

data-science-ml-ai999x 1.0.3, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Data Science and Machine Learning covers evaluation metrics, loss functions and gradients, classical machine-learning algorithms, dimensionality reduction, cross-validation, information theory and neural-network arithmetic. This guide is about `data-science-ml`, the model as a self-contained program for Windows and Linux: 107 capabilities in 14 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/data-science-ml-ai999x/); the Python package (`pip install data-science-ml-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS; the newest version is always under https://www.ai999x.com/downloads/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Data Science and Machine Learning	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-data-science-ml-1.4.0-windows-x86_64.zip
Desktop app: Data Science and Machine Learning	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-data-science-ml-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Data Science and Machine Learning in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-data-science-ml-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-data-science-ml.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-data-science-ml-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-data-science-ml-1.4.0-linux-x86_64.tar.gz
./ai999x-data-science-ml-1.4.0-linux-x86_64/ai999x-data-science-ml
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `data-science-ml` is this one. The checksum line should print OK.

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
data-science-ml --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";$D", "User")
data-science-ml --version
```

In your browser

Open https://www.ai999x.com/models/data-science-ml-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install data-science-ml-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ data-science-ml --version
data-science-ml-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ data-science-ml ask "Binary metrics from counts: TP = 40, FP = 10, FN = 5, TN = 45"
Read as: Binary classification metrics from confusion counts
accuracy = 0.85
precision = 0.8
recall = 0.8888888889
specificity = 0.8181818182
f1 = 0.8421052632
npv = 0.9
balanced_accuracy = 0.8535353535
fpr = 0.1818181818
fnr = 0.1111111111
jaccard = 0.7272727273
  by: Binary classification metrics from confusion counts  (exact rational arithmetic on the four counts)
  assumes: counts are whole numbers >= 0; every ratio is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0)
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ data-science-ml search "binary classification metrics from confusion counts"
ml_binary_metrics_from_counts        Binary classification metrics from confusion counts  (classification metrics)
ml_binary_confusion_matrix           Binary confusion matrix from labels  (classification metrics)
ml_threshold_metrics                 Confusion counts, precision and recall at a score threshold  (classification metrics)
ml_multiclass_confusion_matrix       Multi-class confusion matrix  (classification metrics)
ml_log_loss_binary                   Binary log loss (cross-entropy)  (classification metrics)
ml_focal_loss                        Binary focal loss and its gradient  (loss functions and activations)
ml_bce_with_logits                   Binary cross-entropy with logits and its gradient  (loss functions and activations)
ml_bernoulli_naive_bayes             Bernoulli naive Bayes prediction (binary features)  (classical algorithms)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ data-science-ml list
ml_adam_step                            Adam optimizer: one update
ml_adamw_step                           AdamW optimizer (decoupled weight decay): one update
ml_adjusted_mutual_information          Adjusted mutual information of two clusterings
ml_adjusted_r2                          Adjusted R^2
ml_average_precision                    Average precision and precision-recall AUC
ml_backpropagation                      Backpropagation gradients of a one-hidden-layer network
ml_balanced_accuracy                    Balanced accuracy (multi-class)
ml_balanced_class_weights               Balanced class weights
… 99 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ data-science-ml info ml_binary_metrics_from_counts
Binary classification metrics from confusion counts
  exact rational arithmetic on the four counts
  in   TP       true positives (integer) [1]
  in   FP       false positives (integer) [1]
  in   FN       false negatives (integer) [1]
  in   TN       true negatives (integer) [1]
  out  accuracy (TP + TN) / n [1]
  out  precision TP / (TP + FP) [1]
  out  recall   TP / (TP + FN), the sensitivity [1]
  out  specificity TN / (TN + FP) [1]
  out  f1       2 TP / (2 TP + FP + FN) [1]
  out  npv      TN / (TN + FN) [1]
  out  balanced_accuracy (recall + specificity) / 2 [1]
  out  fpr      FP / (FP + TN) [1]
  out  fnr      FN / (FN + TP) [1]
  out  jaccard  TP / (TP + FP + FN) [1]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ data-science-ml solve ml_binary_metrics_from_counts TP=40 FP=10 FN=5 TN=45
accuracy = 0.85
precision = 0.8
recall = 0.8888888889
specificity = 0.8181818182
f1 = 0.8421052632
npv = 0.9
balanced_accuracy = 0.8535353535
fpr = 0.1818181818
fnr = 0.1111111111
jaccard = 0.7272727273
  by: Binary classification metrics from confusion counts  (exact rational arithmetic on the four counts)
  assumes: counts are whole numbers >= 0; every ratio is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0)
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ data-science-ml solve ml_binary_metrics_from_counts TP=40 FP=10 FN=5 TN=45 --json
{"outcome":"SOLVED","message":"","value":0.842105263158,"unit":"","solved_for":"f1","values":[["accuracy",0.85,""],
["precision",0.8,""],["recall",0.888888888889,""],["specificity",0.818181818182,""],["f1",0.842105263158,""],["npv",0.9,""],
["balanced_accuracy",0.853535353535,""],["fpr",0.181818181818,""],["fnr",0.111111111111,""],
["jaccard",0.727272727273,""]],"target":"f1","phrase":"(TP + TN) / n","assumptions":["counts are whole numbers >= 0","every ratio
is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0)"],"name":"Binary classification metrics from confusion
counts","equation":"exact rational arithmetic on the four counts"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ data-science-ml solve ml_binary_metrics_from_counts TP=40 FP=10 FN=5
NEEDS_INPUT: ml_binary_metrics_from_counts needs TN (true negatives). Example: solve('ml_binary_metrics_from_counts', TP=40,
FP=10, FN=5, TN=45).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ data-science-ml figures ml_binary_metrics_from_counts TP=40 FP=10 FN=5 TN=45 --out .
[
  {
    "slug": "ml-binary-metrics-from-counts-confusion",
    "title": "The four counts as a confusion matrix",
    "caption": "The confusion matrix TP = 40, FN = 5, FP = 10, TN = 45 (rows the actual class, columns the predicted); returned F1
= 0.8421. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-confusion.svg"
  },
  {
    "slug": "ml-binary-metrics-from-counts-rates",
    "title": "Rates from a confusion matrix",
    "caption": "The rates the model returns for TP = 40, FP = 10, FN = 5, TN = 45: acc. 0.85, prec. 0.8, recall 0.8889, spec.
0.8182, F1 0.8421, NPV 0.9, bal.acc 0.8535, Jacc. 0.7273. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-rates.svg"
  },
  {
    "slug": "ml-binary-metrics-from-counts-live-1",
    "title": "2 TP / as false negatives varies",
    "caption": "2 TP / as FN runs from 1 to 9, every other input as you entered it (9 solutions). Marked: your FN = 5 gives f1 =
0.8421. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-live-1.svg"
  },
  {
    "slug": "ml-binary-metrics-from-counts-live-2",
    "title": "2 TP / as true positives varies",
    "caption": "2 TP / as TP runs from 20 to 60, every other input as you entered it (21 solutions). Marked: your TP = 40 gives f1
= 0.8421. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-live-2.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | data-science-ml batch

`cases.jsonl`:

```
{"key": "ml_binary_metrics_from_counts", "inputs": {"TP": "40", "FP": "10", "FN": "5", "TN": "45"}}
{"key": "ml_roc_auc", "inputs": {"y_true": "[0, 1, 1, 0, 1]", "scores": "[0.1, 0.9, 0.8, 0.3, 0.2]"}}
```

```
$ data-science-ml batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":0.842105263158,"unit":"","solved_for":"f1","values":[["accuracy",0.85,""],
["precision",0.8,""],["recall",0.888888888889,""],["specificity",0.818181818182,""],["f1",0.842105263158,""],["npv",0.9,""],
["balanced_accuracy",0.853535353535,""],["fpr",0.181818181818,""],["fnr",0.111111111111,""],
["jaccard",0.727272727273,""]],"target":"f1","phrase":"(TP + TN) / n","assumptions":["counts are whole numbers >= 0","every ratio
is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0)"],"name":"Binary classification metrics from confusion
counts","equation":"exact rational arithmetic on the four counts"}
{"outcome":"SOLVED","message":"","value":0.833333333333,"unit":"","solved_for":"auc","values":[["auc",0.833333333333,""],
["gini",0.666666666667,""],["n_pos",3,""],["n_neg",2,""]],"target":"auc","phrase":"area under the ROC curve","assumptions":
["labels 0 or 1","at least one positive and one negative"],"name":"ROC AUC from scores","equation":"exact count over all positive-
negative pairs (Mann-Whitney), ties counted 1/2"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `data-science-ml info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`data-science-ml solve ... --json` prints one answer as one line of JSON. `data-science-ml batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | data-science-ml batch`. `data-science-ml catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["data-science-ml", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`data-science-ml figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.


```
$ data-science-ml figures ml_binary_metrics_from_counts TP=40 FP=10 FN=5 TN=45 --out .
[
  {
    "slug": "ml-binary-metrics-from-counts-confusion",
    "title": "The four counts as a confusion matrix",
    "caption": "The confusion matrix TP = 40, FN = 5, FP = 10, TN = 45 (rows the actual class, columns the predicted); returned F1
= 0.8421. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-confusion.svg"
  },
  {
    "slug": "ml-binary-metrics-from-counts-rates",
    "title": "Rates from a confusion matrix",
    "caption": "The rates the model returns for TP = 40, FP = 10, FN = 5, TN = 45: acc. 0.85, prec. 0.8, recall 0.8889, spec.
0.8182, F1 0.8421, NPV 0.9, bal.acc 0.8535, Jacc. 0.7273. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-rates.svg"
  },
  {
    "slug": "ml-binary-metrics-from-counts-live-1",
    "title": "2 TP / as false negatives varies",
    "caption": "2 TP / as FN runs from 1 to 9, every other input as you entered it (9 solutions). Marked: your FN = 5 gives f1 =
0.8421. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-live-1.svg"
  },
  {
    "slug": "ml-binary-metrics-from-counts-live-2",
    "title": "2 TP / as true positives varies",
    "caption": "2 TP / as TP runs from 20 to 60, every other input as you entered it (21 solutions). Marked: your TP = 40 gives f1
= 0.8421. Your inputs: TP = 40, FP = 10, FN = 5, TN = 45.",
    "file": ".\\ml-binary-metrics-from-counts-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `data_science_ml_ai999x.wasm` (https://www.ai999x.com/compute/rust/data_science_ml_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("data_science_ml_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `data-science-ml info <id>` shows the same inputs and outputs in the terminal.

Classical algorithms (13)

Agglomerative hierarchical clustering merge heights — `ml_hierarchical_clustering`

Agglomerative clustering of points with single, complete, average or Ward linkage (Euclidean distance): the distance at which each of the n - 1 merges happens, in order, and the final merge height.

$$d(k, i \cup j) = \sqrt{\frac{(n_i + n_k)d_{ki}^2 + (n_j + n_k)d_{kj}^2 - n_k d_{ij}^2}{n_i + n_j + n_k}} \ \text{(Ward)}$$

symbol		meaning	SI unit
`X`	in	points: one row per sample (2 to 40)	—
`method`	in	linkage: single, complete, average or ward	—
`heights`	out	merge distances in merge order (ascending)	—
`final_height`	out	height of the last merge	—

Applies when: 2 to 40 points; no tied merge distances (complete, average, ward).

```
$ data-science-ml solve ml_hierarchical_clustering "X=[[0, 0], [0, 1], [4, 0], [4, 1.5], [10, 10]]" method=single
heights = [1.0,1.5,4.0,10.4043260233]
final_height = 10.40432602
  by: Agglomerative hierarchical clustering merge heights  (repeatedly merge the closest pair; Lance-Williams updates of the
cluster distances)
  method = single
  assumes: 2 to 40 points; no tied merge distances (complete, average, ward)
```

What the worked example means: This calculation determines the hierarchical structure of the data by repeatedly merging the closest clusters. Using the specified linkage method, it records the distance at which each merge occurs, resulting in a sequence of heights [1, 1.5, 4, 10.4043]. The last value in this sequence represents the final merge height 10.4043, indicating the maximum dissimilarity required to combine all points into a single cluster.

Bernoulli naive Bayes prediction (binary features) — `ml_bernoulli_naive_bayes`

Fit a Bernoulli naive Bayes classifier on 0/1 features with additive smoothing alpha (absent features count too) and give the predicted class and posteriors of a query vector.

$$P(x \mid c) = \prod_j p_{cj}^{x_j}(1 - p_{cj})^{1 - x_j}$$

symbol		meaning	SI unit
`X`	in	training features (0 or 1)	—
`y`	in	training labels (whole numbers)	—
`x`	in	query features (0 or 1)	—
`alpha`	in	additive smoothing (> 0)	—
`prediction`	out	class with the largest posterior	—
`classes`	out	classes in sorted order	—
`posteriors`	out	posterior probability of each class	—

Applies when: features 0 or 1; alpha > 0; at least two classes.

```
$ data-science-ml solve ml_bernoulli_naive_bayes "X=[[1, 0, 1], [1, 1, 0], [0, 1, 1], [0, 0, 1], [1, 1, 1]]" "y=[0, 0, 1, 1, 0]"
"x=[1, 0, 0]" alpha=1
prediction = 0
classes = [0,1]
posteriors = [0.860022396417,0.139977603583]
  by: Bernoulli naive Bayes prediction (binary features)  (p_cj = (N_cj + alpha)/(N_c + 2 alpha); log prior + sum of log p or
log(1 - p))
  alpha = 1
  assumes: features 0 or 1; alpha > 0; at least two classes
```

What the worked example means: This calculation determines the most likely class for a set of binary features using Bernoulli naive Bayes. It computes the likelihood of observing the specific pattern of present and absent features for each class, incorporating additive smoothing to handle zero counts. The final prediction is the class label 0 that maximizes the posterior probability, while [0.860022, 0.139978] lists the normalized probabilities for all classes in sorted order.

Best decision-tree split (classification, Gini or entropy) — `ml_decision_stump_classification`

The best single split x_j <= t of a classification tree's root by Gini impurity or entropy (bits): the feature, the threshold (midway between consecutive values), the impurity decrease and the sizes of both sides.

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

symbol		meaning	SI unit
`X`	in	features: one row per sample (whole numbers)	—
`y`	in	class labels (whole numbers)	—
`criterion`	in	split criterion: gini or entropy	—
`feature`	out	index (0-based column of X) of the best feature	—
`threshold`	out	threshold t (go left when x_j <= t)	—
`impurity_decrease`	out	parent impurity minus the size-weighted child impurities	—
`n_left`	out	samples going left	—
`n_right`	out	samples going right	—

Applies when: at least two classes; whole-number features; a unique best split.

```
$ data-science-ml solve ml_decision_stump_classification "X=[[2, 7], [3, 1], [10, 8], [11, 2], [4, 9], [12, 0]]" "y=[0, 0, 1, 1,
0, 1]" criterion=gini
feature = 0
threshold = 7
impurity_decrease = 0.5
n_left = 3
n_right = 3
  by: Best decision-tree split (classification, Gini or entropy)  (scan every feature and every midpoint between consecutive
distinct values; exact fractions for Gini)
  criterion = gini
  assumes: at least two classes; whole-number features; a unique best split
```

What the worked example means: This calculation determines the optimal binary partition for a decision tree's root node by scanning all features and midpoints between distinct values. Using the specified criterion, it computes the impurity decrease to identify the split that most effectively separates classes. The result indicates that splitting on feature index 0 at threshold 7 provides the maximum gain, dividing the dataset into 3 samples on the left and 3 samples on the right.

Best regression-tree split (squared error) — `ml_decision_stump_regression`

The best single split x_j <= t of a regression tree's root by squared error: the feature, the threshold, the decrease of the mean squared error and the mean target on each side.

$$\Delta = \frac{1}{n}[SSE(S) - SSE(S_L) - SSE(S_R)]$$

symbol		meaning	SI unit
`X`	in	features (whole numbers)	—
`y`	in	targets	—
`feature`	out	index of the best feature	—
`threshold`	out	threshold t (left when x_j <= t)	—
`mse_decrease`	out	decrease of the node's mean squared error	—
`left_mean`	out	mean target on the left	—
`right_mean`	out	mean target on the right	—

Applies when: whole-number features; targets not all equal; a unique best split.

```
$ data-science-ml solve ml_decision_stump_regression "X=[[1], [2], [3], [10], [11], [12]]" "y=[1, 1.2, 0.8, 5, 5.4, 4.6]"
feature = 0
threshold = 6.5
mse_decrease = 4
left_mean = 1
right_mean = 5
  by: Best regression-tree split (squared error)  (scan every feature and midpoint; exact sums of squares)
  assumes: whole-number features; targets not all equal; a unique best split
```

What the worked example means: This calculation determines the optimal single partition for a regression tree root by evaluating every possible midpoint between unique whole-number feature values. It computes the reduction in sum of squared errors achieved by splitting the data into two groups based on the selected feature and threshold. The result indicates that choosing feature 0 and setting the cutoff at 6.5 yields the greatest improvement, decreasing the node's mean squared error by 4. The average target value in the left subset is 1, while the right subset averages 5.

Distances and cosine similarity between two vectors — `ml_distance_metrics`

The Euclidean, Manhattan and Chebyshev distances between two feature vectors, and their cosine similarity and cosine distance.

$$d_2 = \sqrt{\sum_i (x_i - z_i)^2}, \quad \cos\theta = \frac{x \cdot z}{\|x\|\|z\|}$$

symbol		meaning	SI unit
`x`	in	first vector	—
`z`	in	second vector	—
`euclidean`	out	sqrt(sum (x - z)^2)	—
`manhattan`	out	sum |x - z|	—
`chebyshev`	out	max |x - z|	—
`cosine_similarity`	out	x.z / (|x| |z|)	—
`cosine_distance`	out	1 - cosine similarity	—

Applies when: equal lengths; non-zero vectors.

```
$ data-science-ml solve ml_distance_metrics "x=[1, 2, 3]" "z=[4, 6, 3]"
euclidean = 5
manhattan = 7
chebyshev = 4
cosine_similarity = 0.8554823885
cosine_distance = 0.1445176115
  by: Distances and cosine similarity between two vectors  (direct sums with fsum)
  assumes: equal lengths; non-zero vectors
```

What the worked example means: This calculation determines how similar or dissimilar two feature vectors are by computing multiple metrics. The Euclidean distance, represented as 5, measures the straight-line geometric gap between the points defined by the input vectors. The Manhattan distance, shown as 7, sums the absolute differences along each dimension, reflecting a grid-like path. The Chebyshev distance, given as 4, captures the maximum difference in any single coordinate. Meanwhile, the cosine similarity of 0.855482 evaluates the angle between the vectors, indicating directional alignment regardless of magnitude, while the cosine distance of 0.144518 quantifies this angular divergence.

Gaussian naive Bayes prediction — `ml_gaussian_naive_bayes`

Fit a Gaussian naive Bayes classifier (class frequencies as priors, per-class feature means and population variances) and give the predicted class and posterior probabilities of a query point.

$$P(c \mid x) \propto P(c) \prod_j \mathcal{N}(x_j;\ \mu_{cj}, \sigma_{cj}^2)$$

symbol		meaning	SI unit
`X`	in	training features: one row per sample	—
`y`	in	training labels (whole numbers)	—
`x`	in	query point	—
`prediction`	out	class with the largest posterior	—
`classes`	out	classes in sorted order	—
`posteriors`	out	posterior probability of each class	—

Applies when: at least two classes; positive within-class variance of every feature.

```
$ data-science-ml solve ml_gaussian_naive_bayes "X=[[6, 180, 12], [5.92, 190, 11], [5.58, 170, 12], [5.92, 165, 10], [5, 100, 6], [5.5, 150, 8], [5.42, 130, 7], [5.75, 150, 9]]" "y=[0, 0, 0, 0, 1, 1, 1, 1]" "x=[6, 130, 8]"
prediction = 1
classes = [0,1]
posteriors = [1.54428667821e-7,0.999999845571]
  by: Gaussian naive Bayes prediction  (log prior + sum of Gaussian log densities (variance divides by n_c), normalised by log-sum-exp)
  assumes: at least two classes; positive within-class variance of every feature
```

What the worked example means: This calculation determines the most likely class for a query point by combining prior class frequencies with Gaussian likelihoods derived from training data. The model computes unnormalized log posteriors using the sum of log priors and per-feature Gaussian densities, then normalizes these values to produce final probabilities. The output identifies the class with the highest posterior probability as 1, while listing all classes in sorted order as [0, 1] and their corresponding probabilities as [1.54429e-07, 1].

k-means clustering from given initial centroids — `ml_kmeans`

Lloyd's k-means algorithm run to convergence from stated initial centroids: the final centroids, the cluster of each point and the inertia (within-cluster sum of squares).

$$\min \sum_{k} \sum_{i \in S_k} \|x_i - \mu_k\|^2$$

symbol		meaning	SI unit
`X`	in	points: one row per sample	—
`C`	in	initial centroids: one row per cluster	—
`centroids`	out	final centroids	—
`labels`	out	cluster (0-based row of C) of each point	—
`inertia`	out	sum of squared distances to the final centroids	—

Applies when: no exact distance tie; no empty cluster during the iterations; convergence within 100 iterations.

```
$ data-science-ml solve ml_kmeans "X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]]" "C=[[1, 1], [5, 6]]"
centroids = [[1.25,1.5],[3.9,5.1]]
labels = [0,0,1,1,1,1,1]
inertia = 8.525
  by: k-means clustering from given initial centroids  (alternate exact assignment and mean update until no assignment changes (at most 100 iterations))
  assumes: no exact distance tie; no empty cluster during the iterations; convergence within 100 iterations
```

What the worked example means: This process executes Lloyd's k-means algorithm starting from the provided initial centroids [1 1; 5 6] for the data points [1 1; 1.5 2; 3 4; 5 7; 3.5 5; 4.5 5; 3.5 4.5]. It iteratively assigns each point to its nearest centroid and updates the centroid positions to the mean of their assigned points until assignments stabilize or the iteration limit is reached. The result consists of the final centroid locations [[1.25, 1.5], [3.9, 5.1]], the cluster label for each input point [0, 0, 1, 1, 1, 1, 1], and the total within-cluster sum of squares, known as inertia 8.525, which measures how tightly the points are grouped around their respective centers.

k-nearest-neighbour classification of a query point — `ml_knn_classify`

Classify a query point by majority vote among its k nearest training points (stated metric), with the neighbours and their distances.

$$\hat{y} = \arg\max_{c} \sum_{i \in N_k(x)} \mathbb{1}(y_i = c)$$

symbol		meaning	SI unit
`X`	in	training points: one row per sample	—
`y`	in	training labels (whole numbers)	—
`x`	in	query point	—
`k`	in	number of neighbours	—
`metric`	in	distance: euclidean or manhattan	—
`prediction`	out	majority class among the k neighbours	—
`votes`	out	number of neighbours of that class	—
`neighbors`	out	indices (0-based rows of X) of the k nearest points, ascending	—
`neighbor_distances`	out	their distances, ascending	—

Applies when: no distance tie at the k-th neighbour; no tied vote; 1 <= k <= n.

```
$ data-science-ml solve ml_knn_classify "X=[[1, 1], [2, 1], [4, 3], [5, 4], [1, 2], [6, 5]]" "y=[0, 0, 1, 1, 0, 1]" "x=[3, 2.2]"
k=3 metric=euclidean
prediction = 0
votes = 2
neighbors = [1,2,4]
neighbor_distances = [1.28062484749,1.56204993518,2.00997512422]
  by: k-nearest-neighbour classification of a query point  (exact distances (squared for Euclidean), stable sort, majority vote)
  metric = euclidean
  assumes: no distance tie at the k-th neighbour; no tied vote; 1 <= k <= n
```

What the worked example means: The algorithm identifies the k nearest training points to the query using the specified metric, resulting in neighbor indices [1, 2, 4] and distances [1.28062, 1.56205, 2.00998]. The final classification 0 is determined by a majority vote among these neighbors, where the winning class received 2 votes. This process relies on the assumption that similar data points are located close together in the feature space defined by the chosen distance measure.

k-nearest-neighbour regression of a query point — `ml_knn_regress`

Predict a value at a query point as the mean (uniform) or inverse-distance-weighted mean of the targets of its k nearest training points.

$$\hat{y} = \frac{\sum_{i \in N_k} w_i y_i}{\sum_{i \in N_k} w_i}, \quad w_i = 1 \text{ or } 1/d_i$$

symbol		meaning	SI unit
`X`	in	training points	—
`y`	in	training targets	—
`x`	in	query point	—
`k`	in	number of neighbours	—
`metric`	in	distance: euclidean or manhattan	—
`weights`	in	uniform mean or inverse-distance weights	—
`prediction`	out	predicted value	—
`neighbors`	out	indices of the k nearest points, ascending	—
`neighbor_distances`	out	their distances, ascending	—

Applies when: no distance tie at the k-th neighbour; distance weights: the query is not a training point.

```
$ data-science-ml solve ml_knn_regress "X=[[1], [2], [3], [10]]" "y=[1.5, 2.5, 3, 9]" "x=[2.4]" k=2 metric=euclidean
weights=uniform
prediction = 2.75
neighbors = [1,2]
neighbor_distances = [0.4,0.6]
  by: k-nearest-neighbour regression of a query point  (exact distances, stable sort, (weighted) mean)
  weights = uniform
  metric = euclidean
  assumes: no distance tie at the k-th neighbour; distance weights: the query is not a training point
```

What the worked example means: This calculation determines a continuous output for a specific query point by identifying its closest training examples. The algorithm computes distances between the query and all training points, selects the 2 nearest ones based on the chosen metric, and then aggregates their target values. Depending on the weighting scheme, this aggregation is either a simple arithmetic mean or an inverse-distance weighted mean. The final result, 2.75, represents the local estimate of the function at that location, while the list [1, 2] identifies which training points contributed to this estimate.

Multinomial naive Bayes prediction (word counts) — `ml_multinomial_naive_bayes`

Fit a multinomial naive Bayes classifier on count features (such as word counts) with additive (Laplace/Lidstone) smoothing alpha and give the predicted class and posteriors of a query count vector.

$$\hat{\theta}_{cj} = \frac{N_{cj} + \alpha}{N_c + \alpha d}, \quad \log P(c \mid x) = \log P(c) + \sum_j x_j \log \hat{\theta}_{cj} + \text{const}$$

symbol		meaning	SI unit
`X`	in	training counts: one row per document, one column per term	—
`y`	in	training labels (whole numbers)	—
`x`	in	query counts	—
`alpha`	in	additive smoothing (> 0; 1 = Laplace)	—
`prediction`	out	class with the largest posterior	—
`classes`	out	classes in sorted order	—
`posteriors`	out	posterior probability of each class	—

Applies when: alpha > 0; counts >= 0; at least two classes.

```
$ data-science-ml solve ml_multinomial_naive_bayes "X=[[2, 1, 0, 0], [1, 1, 0, 1], [0, 0, 3, 1], [0, 1, 2, 2]]" "y=[0, 0, 1, 1]"
"x=[1, 0, 1, 0]" alpha=1
prediction = 0
classes = [0,1]
posteriors = [0.529780564263,0.470219435737]
  by: Multinomial naive Bayes prediction (word counts)  (theta_cj = (N_cj + alpha) / (N_c + alpha d); log prior + sum x_j log
theta_cj; log-sum-exp normalisation)
  alpha = 1
  assumes: alpha > 0; counts >= 0; at least two classes
```

What the worked example means: A multinomial naive Bayes classifier was fitted on training counts using additive smoothing. For the query vector, the algorithm computes smoothed probabilities for each term within every class and combines them with the prior to determine the most likely category. The resulting prediction is 0, which corresponds to the highest posterior probability among all classes.

One k-means iteration (assignment and update) — `ml_kmeans_step`

One Lloyd iteration of k-means from given centroids: assign every point to its nearest centroid, give the inertia of that assignment and the updated centroids (the means of the assigned points).

$$c(i) = \arg\min_k \|x_i - \mu_k\|^2, \quad \mu'_k = \frac{1}{|S_k|} \sum_{i \in S_k} x_i$$

symbol		meaning	SI unit
`X`	in	points: one row per sample	—
`C`	in	current (or initial) centroids: one row per cluster	—
`labels`	out	index (0-based row of C) of the nearest centroid of each point	—
`new_centroids`	out	mean of the points assigned to each centroid	—
`inertia`	out	sum of squared distances of the points to their assigned (current) centroids	—

Applies when: no exact distance tie; no empty cluster; 2 <= k <= n.

```
$ data-science-ml solve ml_kmeans_step "X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5], [3.5, 4.5]]" "C=[[1, 1], [5, 6]]"
labels = [0,0,1,1,1,1,1]
new_centroids = [[1.25,1.5],[3.9,5.1]]
inertia = 19.25
  by: One k-means iteration (assignment and update)  (exact squared Euclidean distances; exact means)
  assumes: no exact distance tie; no empty cluster; 2 <= k <= n
```

What the worked example means: This calculation performs one step of the k-means algorithm using the input points X and current centroids C. It assigns each point to its nearest centroid, resulting in cluster labels [0, 0, 1, 1, 1, 1, 1]. The total squared distance from points to their assigned centroids is reported as inertia 19.25. New centroids are computed as the mean of the points in each cluster, yielding [[1.25, 1.5], [3.9, 5.1]].

Polynomial kernel — `ml_polynomial_kernel`

The polynomial kernel (gamma x.z + coef0)^degree between two vectors, as used by kernel SVMs.

$$k(x, z) = (\gamma\, x^\top z + c_0)^d$$

symbol		meaning	SI unit
`x`	in	first vector	—
`z`	in	second vector	—
`gamma`	in	scale of the dot product	—
`coef0`	in	constant term	—
`degree`	in	degree (1 to 10)	—
`kernel`	out	(gamma x.z + coef0)^degree	—

Applies when: degree 1 to 10; equal lengths.

```
$ data-science-ml solve ml_polynomial_kernel "x=[1, 2]" "z=[3, 4]" gamma=1 coef0=1 degree=2
kernel = 144
  by: Polynomial kernel  (exact rational arithmetic)
  gamma = 1
  coef0 = 1
  assumes: degree 1 to 10; equal lengths
```

What the worked example means: This calculation determines the similarity between two vectors using the polynomial kernel formula. It takes the dot product of vector x and vector z, scales it by gamma, adds the constant term coef0, and raises the result to the power of degree. The final output, 144, represents this computed similarity score.

RBF (Gaussian) kernel matrix — `ml_rbf_kernel_matrix`

The Gram matrix of the radial basis function kernel exp(-gamma ||x_i - x_j||^2) for a set of points, and the mean off-diagonal similarity.

$$K_{ij} = \exp\left(-\gamma \left\|x_i - x_j\right\|^2\right)$$

symbol		meaning	SI unit
`X`	in	points: one row per point (at least two)	—
`gamma`	in	kernel width parameter (> 0)	—
`K`	out	kernel matrix K_ij = exp(-gamma ||x_i - x_j||^2)	—
`mean_offdiagonal`	out	mean of the off-diagonal entries	—

Applies when: gamma > 0; at least two points.

```
$ data-science-ml solve ml_rbf_kernel_matrix "X=[[0, 0], [1, 0], [0, 2]]" gamma=0.5
K = [[1.0,0.606530659713,0.135335283237],[0.606530659713,1.0,0.0820849986239],[0.135335283237,0.0820849986239,1.0]]
mean_offdiagonal = 0.2746503139
  by: RBF (Gaussian) kernel matrix  (squared distances with fsum, then exp)
  gamma = 0.5
  assumes: gamma > 0; at least two points
```

What the worked example means: This calculation produces the Gram matrix for the radial basis function kernel using the provided points and gamma parameter. Each entry represents the similarity between two data points, decaying exponentially with their squared distance. The resulting matrix [[1, 0.606531, 0.135335], [0.606531, 1, 0.082085], [0.135335, 0.082085, 1]] contains these pairwise similarities, while the mean off-diagonal value 0.27465 summarizes the average similarity across all distinct point pairs in the dataset.

Classification metrics (20)

Average precision and precision-recall AUC — `ml_average_precision`

Average precision (the step-wise area under the precision-recall curve, sum of (R_k - R_{k-1}) P_k over distinct score thresholds) and the trapezoidal area under the precision-recall curve.

$$\mathrm{AP} = \sum_k (R_k - R_{k-1})\, P_k$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`scores`	in	classifier scores	—
`average_precision`	out	average precision	—
`pr_auc_trapezoid`	out	trapezoidal area under the precision-recall curve, from (recall 0, precision 1)	—

Applies when: labels 0 or 1; at least one positive.

```
$ data-science-ml solve ml_average_precision "y_true=[0, 1, 1, 0, 1]" "scores=[0.1, 0.9, 0.8, 0.3, 0.2]"
average_precision = 0.9166666667
pr_auc_trapezoid = 0.9027777778
  by: Average precision and precision-recall AUC  (precision and recall at every distinct threshold, exact fractions)
  assumes: labels 0 or 1; at least one positive
```

What the worked example means: This calculation determines the average precision and the trapezoidal area under the precision-recall curve based on the true labels [0, 1, 1, 0, 1] and classifier scores [0.1, 0.9, 0.8, 0.3, 0.2]. Average precision is computed as the step-wise sum of precision at each recall level, while the trapezoidal method approximates the area using linear interpolation between points. The resulting average precision value of 0.916667 indicates how well the classifier ranks positive instances above negative ones, with higher values signifying better performance, particularly in imbalanced datasets where accuracy can be misleading.

Averaged precision, recall and F1 (multi-class) — `ml_precision_recall_f1_averaged`

Multi-class precision, recall and F1 averaged over classes: macro (unweighted), weighted (by support) or micro (pooled counts).

$$F_1^{\mathrm{macro}} = \frac{1}{K} \sum_k \frac{2 P_k R_k}{P_k + R_k}$$

symbol		meaning	SI unit
`y_true`	in	true class labels (whole numbers)	—
`y_pred`	in	predicted class labels (whole numbers)	—
`average`	in	averaging: macro, weighted or micro	—
`precision`	out	averaged precision	—
`recall`	out	averaged recall	—
`f1`	out	averaged F1 (macro and weighted: the average of the per-class F1)	—

Applies when: labels are whole numbers; equal lengths.

```
$ data-science-ml solve ml_precision_recall_f1_averaged "y_true=[0, 1, 2, 2, 1, 0, 2, 1]" "y_pred=[0, 2, 2, 2, 1, 0, 1, 1]"
average=macro
precision = 0.7777777778
recall = 0.7777777778
f1 = 0.7777777778
  by: Averaged precision, recall and F1 (multi-class)  (per-class counts over the sorted union of classes, exact fractions; an
undefined per-class ratio counts as 0)
  average = macro
  assumes: labels are whole numbers; equal lengths
```

What the worked example means: This calculation determines the average performance of a classifier across multiple categories by computing precision, recall, and F1 scores for each class individually and then combining them. The resulting macro averaged F1 score is 0.777778, which treats every class equally regardless of its frequency in the dataset. This metric provides a holistic view of model quality, ensuring that poor performance on minority classes is not hidden by strong performance on majority classes, making it suitable for imbalanced datasets where all classes are important.

Balanced accuracy (multi-class) — `ml_balanced_accuracy`

Balanced accuracy, the mean recall over the classes present in the true labels, and its chance-adjusted form.

$$\mathrm{BA} = \frac{1}{K}\sum_{k}\frac{TP_k}{TP_k + FN_k}$$

symbol		meaning	SI unit
`y_true`	in	true class labels (whole numbers)	—
`y_pred`	in	predicted class labels (whole numbers)	—
`balanced_accuracy`	out	mean per-class recall over the true classes	—
`adjusted`	out	(balanced accuracy - 1/K) / (1 - 1/K), K = number of true classes	—

Applies when: at least two true classes; equal lengths.

```
$ data-science-ml solve ml_balanced_accuracy "y_true=[0, 0, 0, 0, 0, 0, 1, 1]" "y_pred=[0, 0, 0, 0, 0, 0, 0, 1]"
balanced_accuracy = 0.75
adjusted = 0.5
  by: Balanced accuracy (multi-class)  (per-class recall as exact fractions, averaged)
  assumes: at least two true classes; equal lengths
```

What the worked example means: This calculation determines the balanced accuracy by computing the mean of per-class recall scores. It uses the true labels and predicted labels to find how well each class was identified independently, then averages these values. The resulting score of 0.75 reflects performance on imbalanced datasets better than standard accuracy, while the chance-adjusted form provides a normalized measure relative to random guessing.

Balanced class weights — `ml_balanced_class_weights`

Class weights that balance an imbalanced training set: n / (K n_c) for each class c.

$$w_c = \frac{n}{K\,n_c}$$

symbol		meaning	SI unit
`y`	in	training labels (whole numbers)	—
`classes`	out	classes in sorted order	—
`weights`	out	weight of each class, n / (K n_c)	—

Applies when: labels are whole numbers.

```
$ data-science-ml solve ml_balanced_class_weights "y=[0, 0, 0, 0, 0, 0, 1, 1]"
classes = [0,1]
weights = [0.666666666667,2.0]
  by: Balanced class weights  (exact fractions n / (K n_c))
  assumes: labels are whole numbers
```

In the worked example, y = [0, 0, 0, 0, 0, 0, 1, 1] go through the equation above and give classes = [0, 1], weights = [0.6666666667, 2]. It applies only when labels are whole numbers: outside that it says what is wrong instead of returning a number.

Binary classification metrics from confusion counts — `ml_binary_metrics_from_counts`

Accuracy, precision, recall (sensitivity), specificity, F1, negative predictive value, balanced accuracy, false positive and false negative rates and the Jaccard index from TP, FP, FN, TN.

$$P = \frac{TP}{TP+FP},\ R = \frac{TP}{TP+FN},\ F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$$

symbol		meaning	SI unit
`TP`	in	true positives	—
`FP`	in	false positives	—
`FN`	in	false negatives	—
`TN`	in	true negatives	—
`accuracy`	out	(TP + TN) / n	—
`precision`	out	TP / (TP + FP)	—
`recall`	out	TP / (TP + FN), the sensitivity	—
`specificity`	out	TN / (TN + FP)	—
`f1`	out	2 TP / (2 TP + FP + FN)	—
`npv`	out	TN / (TN + FN)	—
`balanced_accuracy`	out	(recall + specificity) / 2	—
`fpr`	out	FP / (FP + TN)	—
`fnr`	out	FN / (FN + TP)	—
`jaccard`	out	TP / (TP + FP + FN)	—

Applies when: counts are whole numbers >= 0; every ratio is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0).

```
$ data-science-ml solve ml_binary_metrics_from_counts TP=40 FP=10 FN=5 TN=45
accuracy = 0.85
precision = 0.8
recall = 0.8888888889
specificity = 0.8181818182
f1 = 0.8421052632
npv = 0.9
balanced_accuracy = 0.8535353535
fpr = 0.1818181818
fnr = 0.1111111111
jaccard = 0.7272727273
  by: Binary classification metrics from confusion counts  (exact rational arithmetic on the four counts)
  assumes: counts are whole numbers >= 0; every ratio is defined (TP + FP, TP + FN, TN + FP, TN + FN > 0)
```

What the worked example means: This calculation derives key performance metrics from the confusion matrix counts. With 40 true positives, 10 false positives, 5 false negatives, and 45 true negatives, the system computes ratios such as precision at 0.8 and recall at 0.888889. The F1 score, which balances these two measures, is calculated as 0.842105. These values reflect how well the classifier distinguishes between the two classes, with higher scores generally indicating better separation and fewer errors in both directions.

Binary confusion matrix from labels — `ml_binary_confusion_matrix`

Count true positives, false positives, false negatives and true negatives from true and predicted 0/1 labels, with the accuracy and error rate.

$$\mathrm{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1, 1 = positive)	—
`y_pred`	in	predicted labels (0 or 1)	—
`TP`	out	true positives	—
`FP`	out	false positives	—
`FN`	out	false negatives	—
`TN`	out	true negatives	—
`accuracy`	out	(TP + TN) / n	—
`error_rate`	out	(FP + FN) / n	—

Applies when: labels are 0 or 1; y_true and y_pred have the same length.

```
$ data-science-ml solve ml_binary_confusion_matrix "y_true=[1, 0, 1, 1, 0, 0, 1, 0]" "y_pred=[1, 0, 0, 1, 0, 1, 1, 0]"
TP = 3
FP = 1
FN = 1
TN = 3
accuracy = 0.75
error_rate = 0.25
  by: Binary confusion matrix from labels  (count each (true, predicted) pair; accuracy and error rate exactly as fractions)
  assumes: labels are 0 or 1; y_true and y_pred have the same length
```

What the worked example means: This calculation compares true labels against predicted binary outcomes to determine how often the model was correct. It counts the number of true positives, false positives, false negatives, and true negatives to form a confusion matrix. From these counts, it derives the overall proportion of correct predictions as 0.75 and incorrect predictions as 0.25, providing a clear snapshot of classification performance.

Binary log loss (cross-entropy) — `ml_log_loss_binary`

The mean binary cross-entropy (log loss) of predicted probabilities of the positive class, in nats and in bits.

$$\mathcal{L} = -\frac{1}{n}\sum_i \left[y_i \ln p_i + (1-y_i)\ln(1-p_i)\right]$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`p`	in	predicted probability of class 1 for each sample (0 < p < 1)	—
`log_loss`	out	mean log loss in nats	—
`log_loss_bits`	out	mean log loss in bits	—

Applies when: labels 0 or 1; 0 < p < 1.

```
$ data-science-ml solve ml_log_loss_binary "y_true=[1, 0, 1, 1]" "p=[0.9, 0.2, 0.7, 0.6]"
log_loss = 0.2990011587
log_loss_bits = 0.4313674888
  by: Binary log loss (cross-entropy)  (-mean(y log p + (1 - y) log(1 - p)) with log1p for the negative class)
  assumes: labels 0 or 1; 0 < p < 1
```

What the worked example means: This calculation determines the average binary cross-entropy for a set of predictions. It takes the true labels and predicted probabilities as inputs, applying the negative logarithm to the probability assigned to the correct class for each sample. The resulting value, 0.299001 nats, quantifies the penalty for incorrect classifications, where lower values indicate better alignment between predicted probabilities and actual outcomes. This metric is particularly sensitive to confident errors, heavily penalizing cases where the model assigns a near-zero probability to the true class.

Brier score — `ml_brier_score`

The Brier score, the mean squared difference between predicted probabilities of the positive class and the 0/1 outcomes.

$$\mathrm{BS} = \frac{1}{n}\sum_i (p_i - y_i)^2$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`p`	in	predicted probability of class 1 (0 to 1)	—
`brier`	out	mean squared error of the probabilities	—

Applies when: labels 0 or 1; 0 <= p <= 1.

```
$ data-science-ml solve ml_brier_score "y_true=[1, 0, 1, 1]" "p=[0.9, 0.2, 0.7, 0.6]"
brier = 0.075
  by: Brier score  (exact mean of (p - y)^2)
  assumes: labels 0 or 1; 0 <= p <= 1
```

What the worked example means: This calculation determines the Brier score by averaging the squared differences between each predicted probability of the positive class and its corresponding true label. The result, 0.075, quantifies the accuracy of probabilistic predictions, where lower values indicate better calibration and sharpness relative to the observed events.

Cohen's kappa (unweighted, linear or quadratic) — `ml_cohen_kappa`

Cohen's kappa, the agreement between two raters (or a classifier and the truth) corrected for chance, optionally weighted by the distance between classes.

$$\kappa = 1 - \frac{\sum_{ij} w_{ij} O_{ij}}{\sum_{ij} w_{ij} E_{ij}}, \quad E_{ij} = \frac{r_i c_j}{n}$$

symbol		meaning	SI unit
`y1`	in	labels of the first rater (whole numbers)	—
`y2`	in	labels of the second rater (whole numbers)	—
`weights`	in	disagreement weights: none, linear or quadratic (distance between class positions in sorted order)	—
`kappa`	out	Cohen's kappa	—

Applies when: labels are whole numbers; at least two classes; equal lengths.

```
$ data-science-ml solve ml_cohen_kappa "y1=[0, 1, 2, 2, 1, 0, 2, 1]" "y2=[0, 2, 2, 2, 1, 0, 1, 1]" weights=none
kappa = 0.619047619
  by: Cohen's kappa (unweighted, linear or quadratic)  (kappa = 1 - (weighted observed disagreement) / (weighted chance
disagreement), exact fractions)
  weights = none
  assumes: labels are whole numbers; at least two classes; equal lengths
```

What the worked example means: This calculation determines the weighted or unweighted Cohen's kappa statistic, which quantifies the agreement between two sets of categorical labels while correcting for the probability that they agreed by random chance. The result, 0.619048, reflects how much better than chance the observed agreement was compared to what would be expected if the raters were assigning labels independently based on their marginal distributions. A value near one suggests perfect agreement, zero indicates agreement no better than chance, and negative values imply systematic disagreement.

Confusion counts, precision and recall at a score threshold — `ml_threshold_metrics`

Classify by a score threshold (score >= threshold is positive) and give the confusion counts, precision, recall and accuracy.

$$\hat{y}_i = \mathbb{1}(s_i \geq t)$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`scores`	in	classifier scores	—
`threshold`	in	decision threshold (a score at or above it is positive)	—
`TP`	out	true positives	—
`FP`	out	false positives	—
`FN`	out	false negatives	—
`TN`	out	true negatives	—
`precision`	out	TP / (TP + FP)	—
`recall`	out	TP / (TP + FN)	—
`accuracy`	out	(TP + TN) / n	—

Applies when: labels 0 or 1; at least one score reaches the threshold; at least one positive.

```
$ data-science-ml solve ml_threshold_metrics "y_true=[0, 1, 1, 0, 1]" "scores=[0.1, 0.9, 0.8, 0.3, 0.2]" threshold=0.25
TP = 2
FP = 1
FN = 1
TN = 1
precision = 0.6666666667
recall = 0.6666666667
accuracy = 0.6
  by: Confusion counts, precision and recall at a score threshold  (threshold the scores, count, exact fractions)
  threshold = 0.25
  assumes: labels 0 or 1; at least one score reaches the threshold; at least one positive
```

What the worked example means: Applying a decision rule where scores at or above 0.25 are classified as positive gives the confusion matrix counts. This results in 2 true positives, 1 false positives, 1 false negatives, and 1 true negatives. From these exact fractions, the result is a precision of 0.666667 and a recall of 0.666667, indicating the tradeoff between identifying correct positive predictions and capturing all actual positive cases.

Expected calibration error — `ml_expected_calibration_error`

The expected calibration error (support-weighted mean gap between accuracy and mean confidence over equal-width probability bins) and the maximum calibration error.

$$\mathrm{ECE} = \sum_b \frac{n_b}{n} |\bar{y}_b - \bar{p}_b|$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`p`	in	predicted probability of class 1	—
`n_bins`	in	number of equal-width bins on [0, 1]	—
`ece`	out	expected calibration error	—
`mce`	out	maximum calibration error over non-empty bins	—
`n_nonempty_bins`	out	number of non-empty bins	—

Applies when: labels 0 or 1; 0 <= p <= 1; bins are [b/n, (b+1)/n); a probability on a bin edge goes to the upper bin.

```
$ data-science-ml solve ml_expected_calibration_error "y_true=[1, 0, 1, 1, 0, 0, 1, 0]" "p=[0.92, 0.13, 0.67, 0.81, 0.36, 0.08,
0.44, 0.58]" n_bins=5
ece = 0.14875
mce = 0.36
n_nonempty_bins = 5
  by: Expected calibration error  (bin b = floor(p * n_bins) (p = 1 in the last bin); per bin |mean(y) - mean(p)|, weighted by the
bin's share)
  assumes: labels 0 or 1; 0 <= p <= 1; bins are [b/n, (b+1)/n); a probability on a bin edge goes to the upper bin
```

What the worked example means: This calculation determines the reliability of predicted probabilities by grouping them into equal-width bins. For each bin, it compares the average true label value against the average predicted probability. The expected calibration error, reported as 0.14875, represents the weighted sum of these absolute differences across all non-empty bins. Additionally, the maximum calibration error, given as 0.36, identifies the single bin with the largest discrepancy between confidence and accuracy.

F-beta score — `ml_fbeta_score`

The F-beta score, the weighted harmonic mean of precision and recall in which recall counts beta times as much as precision.

$$F_\beta = \frac{(1+\beta^2)\,PR}{\beta^2 P + R}$$

symbol		meaning	SI unit
`TP`	in	true positives	—
`FP`	in	false positives	—
`FN`	in	false negatives	—
`beta`	in	weight of recall relative to precision (beta > 0; 1 gives F1, 2 favours recall)	—
`precision`	out	TP / (TP + FP)	—
`recall`	out	TP / (TP + FN)	—
`fbeta`	out	F-beta score	—

Applies when: beta > 0; TP > 0.

```
$ data-science-ml solve ml_fbeta_score TP=40 FP=10 FN=5 beta=2
precision = 0.8
recall = 0.8888888889
fbeta = 0.8695652174
  by: F-beta score  (exact rational arithmetic from the counts)
  beta = 2
  assumes: beta > 0; TP > 0
```

What the worked example means: This calculation determines the F-beta score, which serves as a weighted harmonic mean of precision and recall. The result, 0.869565, reflects a balance where recall counts beta times as much as precision. It is computed using the input true positives, false positives, and false negatives to first establish a precision of 0.8 and a recall of 0.888889. This single metric provides a flexible summary of classification performance tailored to the specific weighting parameter provided.

Hamming loss and subset accuracy (multi-label) — `ml_hamming_loss`

For multi-label predictions (0/1 indicator matrices): the Hamming loss (fraction of wrong labels) and the subset accuracy (fraction of samples with every label right).

$$\mathrm{HL} = \frac{1}{nL}\sum_{i,j}\mathbb{1}(Y_{ij} \neq \hat{Y}_{ij})$$

symbol		meaning	SI unit
`Y_true`	in	true label indicators: one row per sample, one column per label (0 or 1)	—
`Y_pred`	in	predicted label indicators, same shape	—
`hamming_loss`	out	wrong label entries / (samples x labels)	—
`subset_accuracy`	out	fraction of rows predicted exactly	—
`n_mismatches`	out	number of wrong label entries	—

Applies when: entries 0 or 1; same shape.

```
$ data-science-ml solve ml_hamming_loss "Y_true=[[1, 0, 1], [0, 1, 0]]" "Y_pred=[[1, 0, 0], [0, 1, 0]]"
hamming_loss = 0.1666666667
subset_accuracy = 0.5
n_mismatches = 1
  by: Hamming loss and subset accuracy (multi-label)  (count mismatching entries and exactly matching rows)
  assumes: entries 0 or 1; same shape
```

What the worked example means: This calculation evaluates multi-label classification performance by comparing true and predicted binary indicator matrices. The Hamming loss represents the fraction of incorrect labels across all samples, while subset accuracy measures the

proportion of rows where every label prediction matches the truth exactly. With 1 total mismatches identified between the inputs, the resulting metrics provide a clear view of both individual label error rates and overall row-level correctness.

Matthews correlation coefficient — `ml_matthews_correlation`

The Matthews correlation coefficient (phi coefficient) of a binary confusion matrix, a balanced measure between -1 and 1.

$$\mathrm{MCC} = \frac{TP\,TN - FP\,FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

symbol		meaning	SI unit
`TP`	in	true positives	—
`FP`	in	false positives	—
`FN`	in	false negatives	—
`TN`	in	true negatives	—
`mcc`	out	Matthews correlation coefficient	—

Applies when: no empty row or column of the confusion matrix.

```
$ data-science-ml solve ml_matthews_correlation TP=40 FP=10 FN=5 TN=45
mcc = 0.7035264707
  by: Matthews correlation coefficient  ((TP TN - FP FN) / sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN)) with exact integers under the root)
  assumes: no empty row or column of the confusion matrix
```

What the worked example means: This calculation determines the Matthews correlation coefficient using true positives, false positives, false negatives, and true negatives. The resulting value of 0.703526 serves as a balanced measure for binary classification quality, ranging from negative one to one, where higher values indicate better prediction performance regardless of class distribution sizes.

Multi-class confusion matrix — `ml_multiclass_confusion_matrix`

The confusion matrix of a multi-class classifier (true classes in rows, predicted in columns) with the number correct and the accuracy.

$$C_{ij} = \#\{n : y_n = c_i,\ \hat{y}_n = c_j\}$$

symbol		meaning	SI unit
`y_true`	in	true class labels (whole numbers)	—
`y_pred`	in	predicted class labels (whole numbers)	—
`classes`	out	classes in sorted order (the union of both label lists)	—
`matrix`	out	confusion matrix: row = true class, column = predicted class	—
`n_correct`	out	number of correct predictions (the trace)	—
`accuracy`	out	fraction correct	—

Applies when: labels are whole numbers; equal lengths.

```
$ data-science-ml solve ml_multiclass_confusion_matrix "y_true=[0, 1, 2, 2, 1, 0, 2, 1]" "y_pred=[0, 2, 2, 2, 1, 0, 1, 1]"
classes = [0,1,2]
matrix = [[2,0,0],[0,2,1],[0,1,2]]
n_correct = 6
accuracy = 0.75
  by: Multi-class confusion matrix  (count each (true, predicted) pair over the sorted union of classes)
  assumes: labels are whole numbers; equal lengths
```

What the worked example means: This calculation constructs a multi-class confusion matrix by counting how often each true label matches each predicted label. The resulting table, where rows represent actual classes and columns represent predicted classes, reveals specific error patterns beyond simple correctness. With an overall accuracy of 0.75, the trace value of 6 indicates the number of instances where the prediction exactly matched the ground truth, while off-diagonal entries highlight which classes are frequently confused with one another.

Multi-class log loss (categorical cross-entropy) — `ml_log_loss_multiclass`

The mean categorical cross-entropy of predicted class probabilities (rows of P), with the perplexity exp(loss).

$$\mathcal{L} = -\frac{1}{n}\sum_i \ln P_{i,y_i}$$

symbol		meaning	SI unit
`y_true`	in	true class index of each sample (0 to K-1)	—
`P`	in	predicted probabilities: one row per sample, one column per class, rows sum to 1	—
`log_loss`	out	mean log loss in nats	—
`perplexity`	out	exp(log loss)	—

Applies when: labels are column indices of P; rows of P sum to 1; P[i, y_i] > 0.

```
$ data-science-ml solve ml_log_loss_multiclass "y_true=[0, 2, 1]" "P=[[0.7, 0.2, 0.1], [0.1, 0.3, 0.6], [0.25, 0.5, 0.25]]"
log_loss = 0.5202159161
perplexity = 1.682390866
  by: Multi-class log loss (categorical cross-entropy)  (-mean of ln P[i, y_i])
  assumes: labels are column indices of P; rows of P sum to 1; P[i, y_i] > 0
```

What the worked example means: This calculation determines the multi-class log loss by taking the negative mean of the natural logarithm of the predicted probabilities assigned to the true class for each sample. The resulting value, expressed as 0.520216, represents the average surprise or error in nats. A lower value indicates that the model's predicted probability distributions align more closely with the actual labels. Additionally, the exponential of this loss yields the perplexity, shown here as 1.68239, which provides an interpretable measure of how many classes on average the model effectively considers for each prediction.

Per-class precision, recall, F1 and support — `ml_classification_report`

A classification report: precision, recall, F1 and support for every class, with the macro-averaged F1.

$$P_k = \frac{TP_k}{TP_k + FP_k}, \quad R_k = \frac{TP_k}{TP_k + FN_k}$$

symbol		meaning	SI unit
`y_true`	in	true class labels (whole numbers)	—
`y_pred`	in	predicted class labels (whole numbers)	—
`classes`	out	classes in sorted order	—
`precision`	out	precision of each class	—
`recall`	out	recall of each class	—
`f1`	out	F1 of each class	—
`support`	out	number of true members of each class	—
`macro_f1`	out	mean of the per-class F1	—

Applies when: labels are whole numbers; equal lengths.

```
$ data-science-ml solve ml_classification_report "y_true=[0, 1, 2, 2, 1, 0, 2, 1]" "y_pred=[0, 2, 2, 2, 1, 0, 1, 1]"
classes = [0,1,2]
precision = [1.0,0.666666666667,0.666666666667]
recall = [1.0,0.666666666667,0.666666666667]
f1 = [1.0,0.666666666667,0.666666666667]
support = [2,3,3]
macro_f1 = 0.7777777778
  by: Per-class precision, recall, F1 and support  (per-class counts, exact fractions; an undefined ratio counts as 0)
  assumes: labels are whole numbers; equal lengths
```

What the worked example means: This calculation produces a detailed performance breakdown for each distinct class found in the data. It computes precision, recall, and the harmonic mean known as F1 for every label, alongside the support count which indicates how many true instances belong to that class. The result includes a macro-averaged F1 score, representing the unweighted mean of the individual class F1 scores. For the specific class identified as [0, 1, 2], the precision is [1, 0.666667, 0.666667], meaning this fraction of predicted positives were actually correct, while the recall is [1, 0.666667, 0.666667], indicating this fraction of actual positives were successfully detected. The overall balanced performance across all categories is summarized by the macro F1 of 0.777778.

ROC AUC from scores — `ml_roc_auc`

The area under the ROC curve from 0/1 labels and classifier scores: the probability that a random positive scores above a random negative (ties count one half), with the Gini coefficient 2 AUC - 1.

$$\mathrm{AUC} = \frac{1}{n_+ n_-} \sum_{i \in +} \sum_{j \in -} \left[\mathbb{1}(s_i > s_j) + \tfrac{1}{2} \mathbb{1}(s_i = s_j) \right]$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`scores`	in	classifier scores (higher = more likely positive)	—
`auc`	out	area under the ROC curve	—
`gini`	out	2 AUC - 1	—
`n_pos`	out	number of positives	—
`n_neg`	out	number of negatives	—

Applies when: labels 0 or 1; at least one positive and one negative.

```
$ data-science-ml solve ml_roc_auc "y_true=[0, 1, 1, 0, 1]" "scores=[0.1, 0.9, 0.8, 0.3, 0.2]"
auc = 0.8333333333
gini = 0.6666666667
n_pos = 3
n_neg = 2
  by: ROC AUC from scores  (exact count over all positive-negative pairs (Mann-Whitney), ties counted 1/2)
  assumes: labels 0 or 1; at least one positive and one negative
```

What the worked example means: This calculation determines the Area Under the Receiver Operating Characteristic curve by comparing every positive instance against every negative instance using their classifier scores. The result, 0.833333, represents the probability that a random positive case receives a higher score than a random negative case, treating ties as half-correct. This metric relies on having at least one positive label (3) and one negative label (2) in the true labels to be valid.

ROC curve points — `ml_roc_curve`

The ROC curve of a scored classifier: the false and true positive rates at every distinct score threshold (a score at or above the threshold is positive), and the trapezoidal area under it.

$$\mathrm{TPR}(t) = \frac{TP(t)}{P},\ \mathrm{FPR}(t) = \frac{FP(t)}{N}$$

symbol		meaning	SI unit
`y_true`	in	true labels (0 or 1)	—
`scores`	in	classifier scores	—
`thresholds`	out	distinct scores, descending (one per curve point after the origin)	—
`fpr`	out	false positive rates, starting at 0	—
`tpr`	out	true positive rates, starting at 0	—
`auc`	out	trapezoidal area under the curve	—

Applies when: labels 0 or 1; at least one positive and one negative.

```
$ data-science-ml solve ml_roc_curve "y_true=[0, 1, 1, 0, 1]" "scores=[0.1, 0.9, 0.8, 0.3, 0.2]"
thresholds = [0.9,0.8,0.3,0.2,0.1]
fpr = [0.0,0.0,0.0,0.5,0.5,1.0]
tpr = [0.0,0.333333333333,0.666666666667,0.666666666667,1.0,1.0]
auc = 0.8333333333
  by: ROC curve points  (sort by score, accumulate TP and FP per distinct score, exact fractions)
  assumes: labels 0 or 1; at least one positive and one negative
```

What the worked example means: This calculation determines the Receiver Operating Characteristic curve by sorting classifier scores [0.1, 0.9, 0.8, 0.3, 0.2] and comparing them to true labels [0, 1, 1, 0, 1]. It computes the true positive rate and false positive rate at every distinct score threshold, resulting in a set of points defined by thresholds [0.9, 0.8, 0.3, 0.2, 0.1], false positive rates [0, 0, 0, 0.5, 0.5, 1], and true positive rates [0, 0.333333, 0.666667, 0.666667, 1, 1]. The trapezoidal area under this curve, denoted as 0.833333, quantifies the overall ability of the model to distinguish between the two classes across all possible decision boundaries.

Top-k accuracy — `ml_top_k_accuracy`

The fraction of samples whose true class is among the k highest-scored classes.

$$\text{top-}k = \frac{1}{n}\sum_{i} \mathbb{1}\left[\mathrm{rank}_i(y_i) \le k\right]$$

symbol		meaning	SI unit
`y_true`	in	true class index of each sample (0 to K-1)	—
`S`	in	class scores: one row per sample, one column per class (K >= 3)	—
`k`	in	number of top classes that count as a hit	—
`top_k_accuracy`	out	fraction of samples with the true class in the top k	—
`n_hits`	out	number of such samples	—

Applies when: K >= 3 classes; 1 <= k < K; distinct scores within each row.

```
$ data-science-ml solve ml_top_k_accuracy "y_true=[0, 1, 2, 2]" "S=[[0.5, 0.2, 0.2001], [0.3, 0.4, 0.2], [0.2, 0.4, 0.3], [0.7,
0.2, 0.1]]" k=2
top_k_accuracy = 0.75
n_hits = 3
  by: Top-k accuracy  (rank of the true class = number of classes scored strictly higher; a hit when rank < k)
  assumes: K >= 3 classes; 1 <= k < K; distinct scores within each row
```

What the worked example means: Top-k accuracy measures the fraction of samples where the correct class appears among the k highest-scoring predictions. For this dataset, the result is 0.75. This indicates that for 3 out of the total samples, the model assigned one of the top 2 scores to the true label. This metric is useful when exact ranking matters less than whether the correct answer is considered a plausible option by the model.

Clustering metrics (7)

Adjusted mutual information of two clusterings — `ml_adjusted_mutual_information`

The adjusted mutual information between two clusterings: the mutual information corrected for chance by its expected value under random labellings with the same cluster sizes (hypergeometric model).

$$\mathrm{AMI} = \frac{I - E[I]}{\mathrm{mean}(H(U), H(V)) - E[I]}$$

symbol		meaning	SI unit
`labels_true`	in	reference labels (whole numbers)	—
`labels_pred`	in	cluster labels (whole numbers)	—
`method`	in	normaliser: arithmetic, geometric, min or max of the two entropies	—
`ami`	out	(I - E[I]) / (normaliser - E[I])	—
`expected_mi`	out	E[I] in nats	—
`mutual_information`	out	I in nats	—

Applies when: each labelling has at least two clusters.

```
$ data-science-ml solve ml_adjusted_mutual_information "labels_true=[0, 0, 1, 1, 2, 2]" "labels_pred=[0, 0, 1, 1, 1, 1]"
method=arithmetic
ami = 0.6153846154
expected_mi = 0.266835672
mutual_information = 0.6365141683
  by: Adjusted mutual information of two clusterings  (expected mutual information summed over the hypergeometric distribution of
each cell (log-gamma form))
  method = arithmetic
  assumes: each labelling has at least two clusters
```

What the worked example means: This calculation determines the similarity between two clusterings by correcting for chance agreement. It takes the reference labels and predicted labels to compute the observed mutual information, which is 0.636514. This value is then compared against the expected mutual information under random labeling, calculated as 0.266836. The final result, 0.615385, represents how much better the clustering matches than random chance would allow, using the specified normalization method.

Calinski-Harabasz index — `ml_calinski_harabasz_index`

The Calinski-Harabasz (variance ratio) index of a clustering: between-cluster over within-cluster dispersion, scaled by (n - K)/(K - 1) (higher is better).

$$CH = \frac{\mathrm{tr}(B)}{\mathrm{tr}(W)} \cdot \frac{n - K}{K - 1}$$

symbol		meaning	SI unit
`X`	in	points: one row per sample	—
`labels`	in	cluster label of each point (whole numbers)	—
`calinski_harabasz`	out	CH index	—
`between`	out	between-cluster dispersion B	—
`within`	out	within-cluster dispersion W	—

Applies when: 2 <= number of clusters <= n - 1.

```
$ data-science-ml solve ml_calinski_harabasz_index "X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]]" "labels=[0, 0, 1, 1,
1, 1]"
calinski_harabasz = 14.64550265
between = 28.83333333
within = 7.875
  by: Calinski-Harabasz index  (exact rational sums of squares)
  assumes: 2 <= number of clusters <= n - 1
```

What the worked example means: This calculation determines the quality of a clustering by measuring the ratio of variance between clusters to variance within them. The result, 14.6455, is derived using the total number of samples and clusters. A higher value indicates that clusters are well-separated and compact, suggesting a better-defined structure in the data.

Davies-Bouldin index — `ml_davies_bouldin_index`

The Davies-Bouldin index of a clustering: the mean over clusters of the worst ratio of within-cluster scatter to between-centroid distance (lower is better).

$$DB = \frac{1}{K} \sum_{k} \max_{m \neq k} \frac{S_k + S_m}{\lVert c_k - c_m \rVert}$$

symbol		meaning	SI unit
`X`	in	points: one row per sample	—
`labels`	in	cluster label of each point (whole numbers)	—
`davies_bouldin`	out	Davies-Bouldin index	—

Applies when: 2 <= number of clusters <= n - 1; distinct centroids.

```
$ data-science-ml solve ml_davies_bouldin_index "X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]]" "labels=[0, 0, 1, 1, 1, 1]"
davies_bouldin = 0.3747335771
  by: Davies-Bouldin index  (centroids, mean distance to the centroid as scatter, Euclidean centroid distances)
  assumes: 2 <= number of clusters <= n - 1; distinct centroids
```

What the worked example means: This calculation determines the Davies-Bouldin index, a measure of cluster validity. It computes the mean over all clusters of the maximum ratio between the sum of within-cluster scatters and the distance between their centroids. The result, 0.374734, quantifies how compact and well-separated the groups are; lower values suggest better-defined clusters with tight internal structure and clear boundaries from other groups.

Homogeneity, completeness and V-measure — `ml_homogeneity_completeness_v_measure`

Entropy-based clustering scores against true classes: homogeneity (each cluster holds one class), completeness (each class sits in one cluster) and their weighted harmonic mean, the V-measure.

$$h = 1 - \frac{H(C \mid K)}{H(C)}, \quad c = 1 - \frac{H(K \mid C)}{H(K)}, \quad V_\beta = \frac{(1+\beta)hc}{\beta h + c}$$

symbol		meaning	SI unit
`labels_true`	in	true class labels (whole numbers)	—
`labels_pred`	in	cluster labels (whole numbers)	—
`beta`	in	weight of completeness relative to homogeneity in the V-measure (1 = equal)	—
`homogeneity`	out	1 - H(C | K) / H(C) = I / H(C)	—
`completeness`	out	I / H(K)	—
`v_measure`	out	(1 + beta) h c / (beta h + c)	—

Applies when: each labelling has at least two clusters; beta > 0.

```
$ data-science-ml solve ml_homogeneity_completeness_v_measure "labels_true=[0, 0, 1, 1, 2, 2]" "labels_pred=[0, 0, 1, 1, 1, 1]"
beta=1
homogeneity = 0.5793801643
completeness = 1
v_measure = 0.7336804367
  by: Homogeneity, completeness and V-measure  (mutual information and entropies of the contingency table)
  beta = 1
  assumes: each labelling has at least two clusters; beta > 0
```

What the worked example means: This calculation derives entropy-based clustering scores from the contingency table formed by true class labels and predicted cluster labels. Homogeneity, valued at 0.57938, measures whether each cluster contains members of a single class. Completeness, valued at 1, checks if all members of a given class are assigned to the same cluster. The V-measure, resulting in 0.73368, combines these two metrics into a weighted harmonic mean using the parameter beta set to 1. A score near one indicates a strong alignment between the clustering structure and the true categories.

Normalised mutual information of two clusterings — `ml_normalized_mutual_information`

The normalised mutual information between two clusterings (or a clustering and the true classes), with the stated normaliser of the two entropies, and the mutual information in nats.

$$\mathrm{NMI} = \frac{I(U,V)}{\mathrm{mean}(H(U), H(V))}$$

symbol		meaning	SI unit
`labels_true`	in	reference labels (whole numbers)	—
`labels_pred`	in	cluster labels (whole numbers)	—
`method`	in	normaliser: arithmetic, geometric, min or max of the two entropies	—
`nmi`	out	I / normaliser(H_true, H_pred)	—
`mutual_information`	out	I in nats	—

Applies when: each labelling has at least two clusters.

```
$ data-science-ml solve ml_normalized_mutual_information "labels_true=[0, 0, 1, 1, 2, 2]" "labels_pred=[0, 0, 1, 1, 1, 1]"
method=arithmetic
nmi = 0.7336804367
mutual_information = 0.6365141683
  by: Normalised mutual information of two clusterings  (contingency counts; mutual information and entropies in nats)
  method = arithmetic
  assumes: each labelling has at least two clusters
```

What the worked example means: This calculation determines the similarity between two clusterings by computing their mutual information and dividing it by a chosen normalisation factor based on their entropies. The resulting score, 0.73368, indicates how much information one clustering reveals about the other, with higher values suggesting greater agreement. The underlying mutual information in nats is 0.636514.

Rand index, adjusted Rand index and Fowlkes-Mallows index — `ml_rand_index`

Pair-counting agreement between two clusterings: the Rand index, the adjusted Rand index (corrected for chance) and the Fowlkes-Mallows index.

$$\mathrm{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - \frac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \frac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}}$$

symbol		meaning	SI unit
`labels_true`	in	reference labels (whole numbers)	—
`labels_pred`	in	cluster labels (whole numbers)	—
`rand_index`	out	fraction of sample pairs on which the two labellings agree	—
`adjusted_rand_index`	out	ARI	—
`fowlkes_mallows`	out	TP / sqrt((TP + FP)(TP + FN)) over pairs	—

Applies when: at least one within-cluster pair in each labelling.

```
$ data-science-ml solve ml_rand_index "labels_true=[0, 0, 1, 1, 2, 2]" "labels_pred=[0, 0, 1, 1, 1, 1]"
rand_index = 0.7333333333
adjusted_rand_index = 0.4444444444
fowlkes_mallows = 0.6546536707
  by: Rand index, adjusted Rand index and Fowlkes-Mallows index  (pair counts from the contingency table, exact integers)
  assumes: at least one within-cluster pair in each labelling
```

What the worked example means: This calculation determines the similarity between two data clusterings by analyzing pair counts from their contingency table. The raw fraction of agreeing pairs yields a Rand index of 0.733333. To account for random agreement, the adjusted Rand index is computed as 0.444444. Additionally, the Fowlkes-Mallows index, which measures similarity using geometric means of precision and recall for pairs, results in 0.654654. These metrics rely on integer labels to define within-cluster and cross-cluster relationships among all sample pairs.

Silhouette score — `ml_silhouette_score`

The mean silhouette coefficient of a clustering (Euclidean distance) and the silhouette of each sample: (b - a) / max(a, b), a = mean distance within its cluster, b = mean distance to the nearest other cluster.

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

symbol		meaning	SI unit
`X`	in	points: one row per sample	—
`labels`	in	cluster label of each point (whole numbers)	—
`silhouette`	out	mean silhouette coefficient	—
`samples`	out	silhouette of each sample (0 for a point alone in its cluster)	—

Applies when: 2 <= number of clusters <= n - 1.

```
$ data-science-ml solve ml_silhouette_score "X=[[1, 1], [1.5, 2], [3, 4], [5, 7], [3.5, 5], [4.5, 5]]" "labels=[0, 0, 1, 1, 1, 1]"
silhouette = 0.6138149056
samples = [0.785495870518,0.728161920035,0.287385025256,0.591063503361,0.630078239981,0.660704874448]
  by: Silhouette score  (all pairwise Euclidean distances, per-sample a and b)
  assumes: 2 <= number of clusters <= n - 1
```

What the worked example means: This calculation determines how well each data point fits within its assigned cluster compared to other clusters. For every sample, it computes the average distance to points in its own cluster and the average distance to points in the nearest different cluster using Euclidean metrics. The individual scores are combined into a single value, 0.613815, which ranges from negative one to one. A higher value suggests that clusters are dense and well-separated, while values near zero indicate overlapping boundaries or negative values suggest misassignment.

Dimensionality reduction (1)

Principal component analysis (explained variance and first component) — `ml_pca`

Principal component analysis of a data matrix: the variance explained by every component (sample covariance, n - 1), the explained variance ratios, the first principal direction and the first component score of every sample.

$$\Sigma = \frac{1}{n-1}(X - \bar{X})^{\top}(X - \bar{X}) = V \Lambda V^{\top}$$

symbol		meaning	SI unit
`X`	in	data: one row per sample, one column per feature (more samples than features, at most 8 features)	—
`explained_variance`	out	eigenvalues of the sample covariance, descending	—
`explained_variance_ratio`	out	each eigenvalue / their sum	—
`component_1`	out	first principal direction (unit length, largest-magnitude entry positive)	—
`scores_1`	out	projection of each centred sample on component_1	—

Applies when: n > number of features; full rank; a unique first component.

```
$ data-science-ml solve ml_pca "X=[[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3], [2.3, 2.7], [2, 1.6], [1, 1.1], [1.5,
1.6], [1.1, 0.9]]"
explained_variance = [1.28402771217,0.0490833989383]
explained_variance_ratio = [0.963181314349,0.0368186856514]
component_1 = [0.677873398528,0.735178655544]
scores_1 =
[0.827970186201,-1.77758032528,0.992197494415,0.274210415975,1.67580141864,0.912949103159,-0.0991094374984,-1.1445721638,-0.438046136762,-1.22
  by: Principal component analysis (explained variance and first component)  (exact centring and covariance, cyclic Jacobi eigen-
decomposition)
  assumes: n > number of features; full rank; a unique first component
```

What the worked example means: This analysis decomposes the sample covariance matrix derived from data [2.5 2.4; 0.5 0.7; 2.2 2.9; 1.9 2.2; 3.1 3; 2.3 2.7; 2 1.6; 1 1.1; 1.5 1.6; 1.1 0.9] to identify the primary axis of variation. The eigenvalues, sorted in descending order as [1.28403, 0.0490834], represent the magnitude of variance captured by each orthogonal direction. The ratio [0.963181, 0.0368187] indicates what fraction of the total variability is accounted for by these components. The vector [0.677873, 0.735179] defines the orientation of this dominant direction in feature space, while [0.82797, -1.77758, 0.992197, 0.27421, 1.6758, 0.912949, -0.0991094, -1.14457, -0.438046, -1.22382] projects each original observation onto this new axis, effectively summarizing the dataset's most significant trend.

Gradients (3)

Logistic regression loss and gradient (L2-regularised) — `ml_logistic_loss_gradient`

The mean logistic (binary cross-entropy) loss of a linear model with an L2 penalty (lam / 2) ||w||^2 and its gradient with respect to the weights and the bias.

$$L = \frac{1}{n}\sum_{i}\left[\log(1 + e^{z_i}) - y_i z_i\right] + \frac{\lambda}{2}\|w\|^2, \quad \nabla_w L = \frac{1}{n}X^{\top}(\sigma(z) - y) + \lambda w$$

symbol		meaning	SI unit
`X`	in	feature matrix	—
`y`	in	labels (0 or 1)	—
`w`	in	weights	—
`b`	in	bias	—
`lam`	in	L2 regularisation strength (>= 0; the bias is not penalised)	—
`loss`	out	mean log loss + (lam/2)||w||^2	—
`grad_w`	out	X^T (sigma(Xw + b) - y) / n + lam w	—
`grad_b`	out	mean of sigma(Xw + b) - y	—

Applies when: labels 0 or 1; lam >= 0.

```
$ data-science-ml solve ml_logistic_loss_gradient "X=[[1, 2], [2, 1], [3, 3]]" "y=[0, 1, 1]" "w=[0.5, -0.25]" b=0.1 lam=0
loss = 0.485375599
grad_w = [-0.32406170005,-0.0492576850487]
grad_b = -0.02462884252
  by: Logistic regression loss and gradient (L2-regularised)  (stable log-sum-exp form of the log loss; analytic gradient)
  b = 0.1
  lam = 0
  assumes: labels 0 or 1; lam >= 0
```

What the worked example means: This calculation determines the mean binary cross-entropy loss augmented by an L2 penalty, yielding a total loss of 0.485376. It also computes the analytic gradients for the weights and bias. The weight gradient indicates how much to adjust parameters to minimize this combined objective, while the bias gradient captures the average residual error. These values guide optimization steps in training the model.

Mean squared error loss and gradient of a linear model — `ml_mse_gradient`

The mean squared error of a linear model y_hat = X w + b and its gradient with respect to the weights and the bias.

$$L = \frac{1}{n}\|Xw + b - y\|^2, \quad \nabla_w L = \frac{2}{n} X^\top (Xw + b - y)$$

symbol		meaning	SI unit
`X`	in	feature matrix: one row per sample	—
`y`	in	targets, one per row	—
`w`	in	weights, one per column of X	—
`b`	in	bias (intercept)	—
`loss`	out	mean (X w + b - y)^2	—
`grad_w`	out	dL/dw = (2/n) X^T (X w + b - y)	—
`grad_b`	out	dL/db = (2/n) sum of residuals	—

Applies when: dimensions agree.

```
$ data-science-ml solve ml_mse_gradient "X=[[1, 2], [3, 4], [5, 6]]" "y=[1, 2, 3]" "w=[0.1, 0.2]" b=0
loss = 0.9166666667
grad_w = [-6.46666666667,-8.26666666667]
grad_b = -1.8
  by: Mean squared error loss and gradient of a linear model  (exact rational arithmetic (the loss and gradient are polynomials in
the inputs))
  b = 0
  assumes: dimensions agree
```

What the worked example means: This calculation determines how well a linear model fits the data by computing the average of the squared differences between predicted and actual values. Using the feature matrix [1 2; 3 4; 5 6], targets [1, 2, 3], weights [0.1, 0.2], and bias 0, the resulting loss value is 0.916667. The gradients, which indicate the direction and magnitude for adjusting parameters to reduce error, are [-6.46667, -8.26667] for the weights and -1.8 for the bias.

One gradient-descent step for linear regression — `ml_gradient_descent_step_linear`

One full-batch gradient-descent step on the mean squared error of a linear model: the new weights and bias and the loss before and after the step.

$$w \leftarrow w - \eta \nabla_w L, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

symbol		meaning	SI unit
`X`	in	feature matrix: one row per sample	—
`y`	in	targets, one per row	—
`w`	in	weights, one per column of X	—
`b`	in	bias (intercept)	—
`lr`	in	learning rate (> 0)	—
`w_new`	out	w - lr dL/dw	—
`b_new`	out	b - lr dL/db	—
`loss_before`	out	MSE before the step	—
`loss_after`	out	MSE after the step	—

Applies when: lr > 0; dimensions agree.

```
$ data-science-ml solve ml_gradient_descent_step_linear "X=[[1], [2], [3], [4]]" "y=[2, 4, 6, 8]" "w=[1]" b=1 lr=0.1
w_new = [2.0]
b_new = 1.3
loss_before = 3.5
loss_after = 1.69
  by: One gradient-descent step for linear regression  (exact rational arithmetic)
  b = 1
  lr = 0.1
  assumes: lr > 0; dimensions agree
```

What the worked example means: This calculation performs one full-batch gradient descent step for linear regression. Starting with initial weights [1] and bias 1, it uses a learning rate of 0.1 to update the parameters. The resulting new weights are [2] and the new bias is 1.3. The mean squared error before the update was 3.5, and after the update, it became 1.69. This demonstrates how adjusting the parameters based on the gradient reduces prediction error.

Information theory (7)

Gini impurity decrease of a split — `ml_gini_split`

The Gini impurity of a node, the size-weighted Gini impurity of its children after a split, and the decrease (the CART split criterion).

$$\Delta G = G(S) - \sum_{v} \frac{|S_v|}{|S|} G(S_v)$$

symbol		meaning	SI unit
`y`	in	class labels (whole numbers)	—
`g`	in	child (group) each sample goes to (whole numbers)	—
`gini_parent`	out	Gini impurity before the split	—
`gini_children`	out	weighted Gini impurity of the children	—
`gini_decrease`	out	parent minus children	—

Applies when: at least two groups.

```
$ data-science-ml solve ml_gini_split "y=[0, 0, 0, 1, 1, 1, 1, 0]" "g=[0, 0, 0, 0, 1, 1, 1, 1]"
gini_parent = 0.5
gini_children = 0.375
gini_decrease = 0.125
  by: Gini impurity decrease of a split  (exact fractions)
  assumes: at least two groups
```

What the worked example means: This calculation determines the improvement in purity achieved by dividing the dataset into groups based on class labels. It starts with the parent node's Gini impurity, represented here as 0.5, which measures how mixed the classes are before any division occurs. The method then computes the Gini impurity for each resulting subgroup and combines them using weights proportional to their sample sizes, yielding the total weighted child impurity of 0.375. The final result, 0.125, is the difference between these two values, indicating how much more homogeneous the data has become after the split.

Information gain and gain ratio of a split — `ml_information_gain`

The information gain (in bits) of splitting class labels y by a discrete feature x, with the parent and conditional entropies, the split information and the gain ratio (C4.5).

$$IG = H(Y) - \sum_{v} \frac{|S_v|}{|S|} H(Y \mid X = v), \quad GR = \frac{IG}{H(X)}$$

symbol		meaning	SI unit
`y`	in	class labels (whole numbers)	—
`x`	in	feature value of each sample (whole numbers; each value is one branch)	—
`parent_entropy`	out	H(y) in bits	—
`conditional_entropy`	out	H(y | x) in bits	—
`information_gain`	out	H(y) - H(y | x) in bits	—
`split_info`	out	H(x) in bits	—
`gain_ratio`	out	information gain / split information	—

Applies when: the feature takes at least two values.

```
$ data-science-ml solve ml_information_gain "y=[1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0]" "x=[0, 0, 1, 2, 2, 2, 1, 0, 0, 2, 0, 1,
1, 2]"
parent_entropy = 0.8631205686
conditional_entropy = 0.6324823552
information_gain = 0.2306382134
split_info = 1.577406283
gain_ratio = 0.146213576
  by: Information gain and gain ratio of a split  (entropies of the branches weighted by their sizes)
  assumes: the feature takes at least two values
```

What the worked example means: This calculation determines the reduction in uncertainty about class labels when splitting the dataset by the feature. The parent entropy is 0.863121 bits, while the weighted conditional entropy after the split is 0.632482 bits. The resulting information gain is 0.230638 bits. To account for the number of branches created by the feature, the split information is calculated as 1.57741 bits. Dividing the information gain by this split information produces the gain ratio of 0.146214, which helps compare features with different numbers of distinct values fairly.

Joint, marginal and conditional entropy and mutual information — `ml_joint_conditional_entropy`

From a joint probability table of X (rows) and Y (columns): the joint entropy, both marginal entropies, both conditional entropies and the mutual information.

$$H(Y \mid X) = H(X,Y) - H(X), \quad I(X;Y) = \sum_{ij} P_{ij} \log \frac{P_{ij}}{P_i P_j}$$

symbol		meaning	SI unit
`P`	in	joint probabilities P(X = i, Y = j): rows are values of X, columns values of Y; sums to 1	—
`unit`	in	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	—
`H_XY`	out	joint entropy	—
`H_X`	out	entropy of X	—
`H_Y`	out	entropy of Y	—
`H_Y_given_X`	out	H(Y | X) = H(X, Y) - H(X)	—
`H_X_given_Y`	out	H(X | Y)	—
`mutual_information`	out	I(X; Y) = sum P log P / (P_X P_Y)	—

Applies when: probabilities >= 0 summing to 1.

```
$ data-science-ml solve ml_joint_conditional_entropy "P=[[0.25, 0.25], [0.5, 0]]" unit=bits
H_XY = 1.5
H_X = 1
H_Y = 0.8112781245
H_Y_given_X = 0.5
H_X_given_Y = 0.6887218755
mutual_information = 0.3112781245
  by: Joint, marginal and conditional entropy and mutual information  (entropies of the table and its marginals; mutual
information summed directly)
  unit = bits
  assumes: probabilities >= 0 summing to 1
```

What the worked example means: The provided joint probability table gives the uncertainty inherent in the combined variables. The total joint entropy 1.5 represents the average surprise when observing both X and Y together. The individual uncertainties for each variable are also determined, yielding marginal entropies 1 for X and 0.811278 for Y. The mutual information 0.311278 quantifies how much knowing one variable reduces uncertainty about the other, effectively measuring their statistical dependence. Additionally, the conditional entropies 0.5 and 0.688722 show the remaining uncertainty in one variable after the other is known.

KL divergence, Jensen-Shannon divergence and cross-entropy of discrete distributions — `ml_kl_divergence_discrete`

The Kullback-Leibler divergence of two discrete distributions in both directions, the Jensen-Shannon divergence, the cross-entropy H(p, q) and the entropy H(p).

$$D_{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i}, \quad H(p,q) = H(p) + D_{KL}(p\|q)$$

symbol		meaning	SI unit
`p`	in	first distribution (positive, sums to 1)	—
`q`	in	second distribution (positive, sums to 1)	—
`unit`	in	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	—
`kl_pq`	out	KL(p || q)	—
`kl_qp`	out	KL(q || p)	—
`js_divergence`	out	Jensen-Shannon divergence	—
`cross_entropy`	out	H(p, q) = -sum p log q	—
`entropy_p`	out	H(p)	—

Applies when: positive probabilities summing to 1; same length.

```
$ data-science-ml solve ml_kl_divergence_discrete "p=[0.5, 0.5]" "q=[0.9, 0.1]" unit=bits
kl_pq = 0.7369655942
kl_qp = 0.5310044064
js_divergence = 0.1467931024
cross_entropy = 1.736965594
entropy_p = 1
  by: KL divergence, Jensen-Shannon divergence and cross-entropy of discrete distributions  (direct sums with fsum)
  unit = bits
  assumes: positive probabilities summing to 1; same length
```

What the worked example means: This calculation determines the information difference between two discrete probability distributions. It computes the Kullback-Leibler divergence in both directions, yielding 0.736966 for the first to the second and 0.531004 for the second to the first. Additionally, it provides the symmetric Jensen-Shannon divergence at 0.146793, the cross-entropy of 1.73697, and the entropy of the first distribution at 1. These values quantify how much information is lost when one distribution is used to approximate the other.

Mutual information between two label sequences — `ml_mutual_information_labels`

The mutual information between two discrete variables observed together (two label lists), with both entropies and the conditional entropy H(x | y), from the empirical joint distribution.

$$I(X;Y) = \sum_{ij} \frac{n_{ij}}{n} \log \frac{n\, n_{ij}}{a_i b_j}$$

symbol		meaning	SI unit
`x`	in	values of the first variable (whole numbers)	—
`y`	in	values of the second variable (whole numbers)	—
`unit`	in	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	—
`mutual_information`	out	I(x; y)	—
`H_x`	out	entropy of x	—
`H_y`	out	entropy of y	—
`H_x_given_y`	out	H(x | y) = H(x) - I	—

Applies when: labels are whole numbers; equal lengths.

```
$ data-science-ml solve ml_mutual_information_labels "x=[0, 0, 1, 1, 2, 2]" "y=[0, 0, 1, 1, 1, 1]" unit=bits
mutual_information = 0.9182958341
H_x = 1.584962501
H_y = 0.9182958341
H_x_given_y = 0.6666666667
  by: Mutual information between two label sequences  (contingency table of counts; I = sum n_ij/n log(n n_ij / (a_i b_j)))
  unit = bits
  assumes: labels are whole numbers; equal lengths
```

What the worked example means: This calculation determines the mutual information between two discrete variables, x and y, using their empirical joint distribution. The result, 0.918296, quantifies the shared information between the sequences. Additionally, the entropy of x is 1.58496 and the entropy of y is 0.918296. The conditional entropy H(x given y), which represents the remaining uncertainty in x after observing y, is calculated as 0.666667. These values are derived from a contingency table of counts, ensuring that both input sequences have equal lengths and consist of whole number labels.

Node impurity from class counts (entropy, Gini, misclassification) — `ml_impurity_from_counts`

The impurity of a decision-tree node from its class counts: entropy in bits, Gini impurity and misclassification error.

$$G = 1 - \sum_k p_k^2, \quad H = -\sum_k p_k \log_2 p_k$$

symbol		meaning	SI unit
`counts`	in	number of samples of each class in the node (whole numbers)	—
`entropy_bits`	out	entropy in bits	—
`gini`	out	1 - sum p_k^2	—
`misclassification`	out	1 - max p_k	—

Applies when: counts >= 0 with a positive total.

```
$ data-science-ml solve ml_impurity_from_counts "counts=[9, 5]"
entropy_bits = 0.9402859587
gini = 0.4591836735
misclassification = 0.3571428571
  by: Node impurity from class counts (entropy, Gini, misclassification)  (class proportions; Gini and misclassification exactly
as fractions)
  assumes: counts >= 0 with a positive total
```

In the worked example, counts = [9, 5] go through the equation above and give entropy_bits = 0.9402859587, gini = 0.4591836735, misclassification = 0.3571428571. It applies only when counts >= 0 with a positive total: outside that it says what is wrong instead of returning a number.

Shannon entropy and perplexity of a distribution — `ml_entropy`

The Shannon entropy of a discrete probability distribution in the stated unit, its perplexity (e^H in nats = 2^H in bits), the maximum entropy log K and the normalised entropy H / log K.

$$H(p) = -\sum_i p_i \log p_i$$

symbol		meaning	SI unit
`p`	in	probabilities (>= 0, sum to 1)	—
`unit`	in	unit of information: nats (natural log), bits (log base 2) or bans (log base 10)	—
`entropy`	out	H(p) in the stated unit	—
`perplexity`	out	exp(H in nats)	—
`max_entropy`	out	log K in the stated unit	—

symbol		meaning	SI unit
`normalized_entropy`	out	H / log K (0 for K = 1)	—

Applies when: probabilities >= 0 summing to 1.

```
$ data-science-ml solve ml_entropy "p=[0.5, 0.25, 0.25]" unit=bits
entropy = 1.5
perplexity = 2.828427125
max_entropy = 1.584962501
normalized_entropy = 0.9463946304
  by: Shannon entropy and perplexity of a distribution  (-sum p log p with 0 log 0 = 0 (fsum))
  unit = bits
  assumes: probabilities >= 0 summing to 1
```

What the worked example means: This calculation determines the uncertainty inherent in the provided probability distribution. Applying the formula negative sum of p log p, treating zero probabilities as contributing nothing, gives the entropy value 1.5. This result indicates how unpredictable the distribution is relative to its maximum possible disorder. The associated perplexity, calculated as the exponential of the entropy in nats, yields 2.82843, representing the effective number of equally likely outcomes. Additionally, the normalized entropy compares this uncertainty to the theoretical maximum for the given number of categories.

Linear models (6)

Lasso regression coefficients (stated alpha) — `ml_lasso_regression`

Fit lasso regression: minimise ||y - X w - b||^2 / (2n) + alpha ||w||_1 (the intercept is not penalised); give the coefficients (some exactly zero), the intercept and the number of non-zero coefficients.

$$\min_{w,b} \frac{1}{2n}\|y - Xw - b\|^2 + \alpha\|w\|_1$$

symbol		meaning	SI unit
`X`	in	features: one row per sample	—
`y`	in	targets	—
`alpha`	in	L1 penalty strength (> 0)	—
`w`	out	coefficients	—
`b`	out	intercept	—
`n_nonzero`	out	number of non-zero coefficients	—

Applies when: alpha > 0; non-constant features; a non-degenerate solution.

```
$ data-science-ml solve ml_lasso_regression "X=[[1, 0.5], [2, 1.5], [3, 1], [4, 3], [5, 2], [6, 3.5]]" "y=[1.1, 2.3, 2.9, 4.2,
5.1, 5.8]" alpha=0.1
w = [0.914285714286,0.0]
b = 0.3666666667
n_nonzero = 1
  by: Lasso regression coefficients (stated alpha)  (coordinate descent to find the active set and signs, then the exact solution
on that set; the optimality conditions are checked)
  alpha = 0.1
  assumes: alpha > 0; non-constant features; a non-degenerate solution
```

What the worked example means: Lasso regression minimizes the mean squared error plus an L1 penalty scaled by alpha to produce sparse coefficients. For this dataset, the algorithm found weights [0.914286, 0] and an intercept of 0.366667. The resulting sparsity level is 1 non-zero coefficients, indicating how many features remain in the final linear model after regularization.

Linear SVM decision values, classes and margin — `ml_linear_svm_decision`

For a linear SVM with given weights and bias: the decision value w.x + b of each sample, its class (+1 when the value is positive, else -1), the margin width 2 / ||w|| and each sample's distance to the hyperplane.

$$f(x) = w^\top x + b, \quad \text{margin} = \frac{2}{\|w\|}$$

symbol		meaning	SI unit
`w`	in	weights	—
`b`	in	bias	—
`X`	in	samples: one row each	—
`decision`	out	w.x + b	—
`classes`	out	+1 if w.x + b > 0, else -1	—
`margin_width`	out	2 / ||w||	—
`distances`	out	|w.x + b| / ||w||	—

Applies when: w is not zero.

```
$ data-science-ml solve ml_linear_svm_decision "w=[2, -1]" b=-1 "X=[[1, 0], [0, 1], [2, 2]]"
decision = [1.0,-2.0,1.0]
classes = [1,-1,1]
margin_width = 0.894427191
distances = [0.4472135955,0.894427191,0.4472135955]
  by: Linear SVM decision values, classes and margin  (exact decision values; one square root for ||w||)
  b = -1
  assumes: w is not zero
```

What the worked example means: This calculation determines the linear SVM's output for each sample. The decision value is found by taking the dot product of the weight vector and the sample features, then adding the bias term. Samples with positive decision values are classified as one class, while negative or zero values belong to the other. The geometric margin width is calculated as two divided by the norm of the weight vector, resulting in 0.894427. Additionally, the perpendicular distance from each sample to the separating hyperplane is derived by dividing the absolute decision value by the weight vector norm.

Logistic regression fit (L2, stated C) — `ml_logistic_regression_fit`

Fit binary logistic regression with an L2 penalty: minimise 0.5 ||w||^2 + C sum of log-losses (the intercept is not penalised); give the weights, the intercept, the objective and the training accuracy.

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \log\left(1 + e^{-\tilde{y}_i(w^\top x_i + b)}\right)$$

symbol		meaning	SI unit
`X`	in	features: one row per sample	—
`y`	in	labels (0 and 1)	—
`C`	in	inverse regularisation strength (> 0)	—
`w`	out	fitted weights	—
`b`	out	fitted intercept	—
`objective`	out	minimised objective	—
`train_accuracy`	out	fraction of training samples classified correctly (z > 0 -> 1)	—

Applies when: labels 0 and 1, both present; C > 0; Newton's method converges.

```
$ data-science-ml solve ml_logistic_regression_fit "X=[[0.5], [1], [1.5], [2], [2.5], [3], [3.5], [4]]" "y=[0, 0, 1, 0, 1, 0, 1,
1]" C=1
w = [0.73877634783]
b = -1.662246783
objective = 4.653114315
train_accuracy = 0.75
  by: Logistic regression fit (L2, stated C)  (Newton's method on the strictly convex objective (exact gradient and Hessian) to
machine precision)
  C = 1
  assumes: labels 0 and 1, both present; C > 0; Newton's method converges
```

What the worked example means: This calculation fits a binary logistic regression model using L2 regularization by minimizing the sum of the squared weight norm scaled by one half and the inverse regularization strength C times the total log loss. The optimization uses Newton's method on this strictly convex objective to find the exact gradient and Hessian, yielding the fitted weights [0.738776] and intercept -1.66225. The resulting minimized objective value is 4.65311, indicating how well the model balances fit and complexity, while the training accuracy is 0.75, representing the fraction of samples correctly classified.

Logistic regression prediction from given weights — `ml_logistic_regression_predict`

The logits, predicted probabilities of class 1 and predicted classes of a logistic regression model with given weights and intercept.

$$P(y = 1 \mid x) = \sigma(w^\top x + b)$$

symbol		meaning	SI unit
`w`	in	weights	—
`b`	in	intercept	—
`X`	in	samples to predict: one row each	—
`probabilities`	out	sigma(w.x + b) for each sample	—
`classes`	out	1 when w.x + b > 0, else 0	—
`logits`	out	w.x + b	—

Applies when: w has one weight per feature.

```
$ data-science-ml solve ml_logistic_regression_predict "w=[0.8, -0.4]" b=-0.2 "X=[[1, 2], [3, 1], [0, 0]]"
probabilities = [0.450166002688,0.8581489351,0.450166002688]
classes = [0,1,0]
logits = [-0.2,1.8,-0.2]
  by: Logistic regression prediction from given weights  (stable sigmoid of the linear score)
  b = -0.2
  assumes: w has one weight per feature
```

What the worked example means: This calculation determines the likelihood that an observation belongs to the positive class using logistic regression. It begins by computing a linear combination of the input features and weights, resulting in a logit value of [-0.2, 1.8, -0.2]. This raw score is then transformed through a stable sigmoid function to produce a probability between zero and one, shown here as [0.450166, 0.858149, 0.450166]. Based on this probability, the model assigns a class label of [0, 1, 0], selecting the positive class when the underlying linear score exceeds zero.

Perceptron training epoch — `ml_perceptron_epoch`

One pass of the perceptron learning rule through the samples in order (labels -1/+1): whenever y (w.x + b) <= 0, update w += lr y x and b += lr y; the final weights, bias and number of updates.

$$\text{if } y_i(w^\top x_i + b) \le 0:\ w \leftarrow w + \eta y_i x_i,\ b \leftarrow b + \eta y_i$$

symbol		meaning	SI unit
`X`	in	samples in presentation order	—
`y`	in	labels (-1 or +1)	—
`w`	in	initial weights	—
`b`	in	initial bias	—
`lr`	in	learning rate (> 0)	—
`w_new`	out	weights after the epoch	—
`b_new`	out	bias after the epoch	—
`n_updates`	out	number of updates (mistakes, including y f = 0)	—

Applies when: labels -1 and +1; lr > 0.

```
$ data-science-ml solve ml_perceptron_epoch "X=[[2, 1], [-1, -2], [1, -1], [-2, 1]]" "y=[1, -1, 1, -1]" "w=[0, 0]" b=0 lr=1
w_new = [2.0,1.0]
b_new = 1
n_updates = 1
  by: Perceptron training epoch  (exact rational arithmetic, samples in the given order)
  b = 0
  lr = 1
  assumes: labels -1 and +1; lr > 0
```

What the worked example means: This calculation performs one training epoch of the perceptron learning rule using exact rational arithmetic. It iterates through the provided samples in order, checking if the current classification is incorrect by evaluating whether the product of the label and the weighted sum plus bias is less than or equal to zero. If an error occurs, the weights are adjusted by adding the product of the learning rate, the label, and the input vector, and the bias is similarly updated. The final result reports the new weight vector [2, 1], the new bias term 1, and the total count of corrections applied during this pass, 1.

Ridge regression coefficients (stated alpha) — `ml_ridge_regression`

Fit ridge regression: minimise ||y - X w - b||^2 + alpha ||w||^2 (the intercept is not penalised); give the coefficients, the intercept and the training MSE.

$$\hat{w} = (X_c^\top X_c + \alpha I)^{-1} X_c^\top y_c$$

symbol		meaning	SI unit
`X`	in	features: one row per sample	—
`y`	in	targets	—
`alpha`	in	L2 penalty strength (>= 0; 0 is least squares)	—
`w`	out	coefficients	—
`b`	out	intercept	—
`train_mse`	out	training mean squared error	—

Applies when: alpha >= 0; a non-singular system (alpha > 0 always is).

```
$ data-science-ml solve ml_ridge_regression "X=[[1], [2], [3], [4], [5]]" "y=[1.2, 1.9, 3.1, 3.9, 5.2]" alpha=1
w = [0.909090909091]
b = 0.3327272727
train_mse = 0.03492892562
  by: Ridge regression coefficients (stated alpha)  (centre X and y, solve (Xc^T Xc + alpha I) w = Xc^T yc exactly, b = mean(y) -
mean(X) w)
  alpha = 1
  assumes: alpha >= 0; a non-singular system (alpha > 0 always is)
```

What the worked example means: Ridge regression was performed using a penalty strength of 1 to fit a linear model to the provided data. The resulting coefficient vector is [0.909091], which indicates how much each feature influences the target variable, while the intercept term is calculated as 0.332727. The model's performance on the training set is summarized by a mean squared error of 0.0349289, reflecting the average squared difference between the predicted and actual values.

Loss functions and activations (13)

Binary cross-entropy with logits and its gradient — `ml_bce_with_logits`

The mean binary cross-entropy computed directly from logits (stable for large |z|; soft targets allowed) and its gradient with respect to each logit.

$$\ell(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}), \quad \frac{\partial \ell}{\partial z} = \sigma(z) - y$$

symbol		meaning	SI unit
`y`	in	targets between 0 and 1	—
`z`	in	logits	—
`loss`	out	mean of max(z, 0) - z y + log(1 + e^-|z|)	—
`gradient`	out	dL/dz_i = (sigma(z_i) - y_i) / n	—

Applies when: 0 <= y <= 1.

```
$ data-science-ml solve ml_bce_with_logits "y=[1, 0, 1]" "z=[2, -1, -0.5]"
loss = 0.4714222276
gradient = [-0.0397343073407,0.0896471404567,-0.207486443734]
  by: Binary cross-entropy with logits and its gradient  (the log-sum-exp form max(z, 0) - z y + log1p(e^-|z|))
  assumes: 0 <= y <= 1
```

What the worked example means: This calculation determines the average binary cross-entropy loss directly from raw scores, using a numerically stable formula that prevents overflow for large values. The input targets are constrained between zero and one, while the logits represent unnormalized predictions. The resulting loss value of 0.471422 quantifies the discrepancy between the predicted distribution and the actual target, indicating how well the model classifies the data. Simultaneously, the gradient value of [-0.0397343, 0.0896471, -0.207486] shows the direction and magnitude needed to adjust the logits to reduce this error during training.

Binary focal loss and its gradient — `ml_focal_loss`

The mean binary focal loss -alpha_t (1 - p_t)^gamma log p_t from logits (it down-weights easy examples) and its gradient with respect to each logit.

$$FL = -\alpha_t (1 - p_t)^\gamma \log p_t$$

symbol		meaning	SI unit
`y`	in	labels (0 or 1)	—
`z`	in	logits	—
`gamma`	in	focusing parameter (>= 0; 0 gives weighted cross-entropy)	—
`alpha`	in	weight of the positive class (0 to 1; negatives get 1 - alpha)	—
`loss`	out	mean focal loss	—
`gradient`	out	dL/dz_i	—

Applies when: labels 0 or 1; gamma >= 0; 0 <= alpha <= 1.

```
$ data-science-ml solve ml_focal_loss "y=[1, 0, 1]" "z=[2, -1, -0.5]" gamma=2 alpha=0.25
loss = 0.03726577802
gradient = [-0.000405911682949,0.0131452824944,-0.0438459851562]
  by: Binary focal loss and its gradient  (p_t = sigma(+-z), 1 - p_t = sigma(-+z) (no cancellation); analytic gradient alpha_t (1
- p_t)^gamma (gamma p_t log p_t - (1 - p_t)) with the sign of the label)
  gamma = 2
  alpha = 0.25
  assumes: labels 0 or 1; gamma >= 0; 0 <= alpha <= 1
```

What the worked example means: This calculation determines the binary focal loss and its analytic gradient from the provided labels, logits, focusing parameter, and class weight. The resulting mean loss value of 0.0372658 quantifies the penalty for misclassification while down-

weighting easy examples. The gradient value of [-0.000405912, 0.0131453, -0.043846] indicates the direction and magnitude needed to adjust each logit during optimization.

Cross-entropy loss from logits and its gradient — `ml_cross_entropy_logits`

The cross-entropy loss of one sample from its logits and true class, the predicted probability of that class, and the gradient of the loss with respect to the logits (softmax minus one-hot).

$$L = -\log \frac{e^{z_y}}{\sum_j e^{z_j}}, \quad \frac{\partial L}{\partial z_j} = p_j - \mathbb{1}(j = y)$$

symbol		meaning	SI unit
`z`	in	logits of the classes	—
`y`	in	index of the true class (0 to K-1)	—
`loss`	out	logsumexp(z) - z_y	—
`probability`	out	softmax probability of the true class	—
`gradient`	out	dL/dz = softmax(z) - onehot(y)	—

Applies when: y is a class index.

```
$ data-science-ml solve ml_cross_entropy_logits "z=[2, 1, 0.1]" y=0
loss = 0.4170300163
probability = 0.6590011389
gradient = [-0.340998861114,0.242432970705,0.0985658904093]
  by: Cross-entropy loss from logits and its gradient  (log-sum-exp form; the true-class gradient as minus the sum of the other
probabilities (no cancellation))
  assumes: y is a class index
```

What the worked example means: This calculation determines the cross-entropy loss for a single sample using its raw logits and the index of the true class. The loss value, represented as 0.41703, measures how poorly the model predicts the correct category. The probability assigned to the true class, shown as 0.659001, indicates the model's confidence in that specific prediction. Additionally, the gradient with respect to the logits, denoted as [-0.340999, 0.242433, 0.0985659], reveals how each logit should be adjusted to minimize the loss.

Cross-entropy with label smoothing — `ml_label_smoothing_cross_entropy`

The cross-entropy of logits against a smoothed target (1 - epsilon on the true class plus epsilon spread evenly over all K classes) and its gradient with respect to the logits.

$$q_j = \frac{\varepsilon}{K} + (1 - \varepsilon)\mathbb{1}(j = y), \quad L = -\sum_j q_j \log \sigma(z)_j$$

symbol		meaning	SI unit
`z`	in	logits	—
`y`	in	index of the true class (0 to K-1)	—
`epsilon`	in	smoothing amount (0 <= epsilon < 1)	—
`loss`	out	-sum_j q_j log softmax(z)_j	—
`gradient`	out	softmax(z) - q	—

Applies when: 0 <= epsilon < 1; y is a class index.

```
$ data-science-ml solve ml_label_smoothing_cross_entropy "z=[2, 1, 0.1]" y=0 epsilon=0.1
loss = 0.5136966829
gradient = [-0.274332194447,0.209099637371,0.065232557076]
  by: Cross-entropy with label smoothing  (smoothed target q_j = epsilon/K + (1 - epsilon) 1(j = y); stable log-softmax)
  epsilon = 0.1
  assumes: 0 <= epsilon < 1; y is a class index
```

What the worked example means: This calculation determines the cross-entropy loss using label smoothing, where the target distribution places a probability of one minus epsilon on the true class and spreads epsilon evenly across all classes. Given the input logits [2, 1, 0.1], the true class index 0, and the smoothing parameter 0.1, the resulting loss value is 0.513697. The gradient, which indicates how the loss changes with respect to each logit, is computed as the difference between the softmax probabilities and the smoothed targets, yielding [-0.274332, 0.2091, 0.0652326]. This method helps prevent the model from becoming overly confident in its predictions.

Hinge and squared hinge loss — `ml_hinge_loss`

The mean hinge loss max(0, 1 - y f) of a margin classifier with labels -1/+1, the mean squared hinge loss and the number of margin violations.

$$\ell(y, f) = \max(0,\ 1 - yf)$$

symbol		meaning	SI unit
`y_true`	in	true labels (-1 or +1)	—

symbol		meaning	SI unit
`scores`	in	decision values f(x)	—
`hinge`	out	mean max(0, 1 - y f)	—
`squared_hinge`	out	mean max(0, 1 - y f)^2	—
`n_violations`	out	samples with y f < 1	—

Applies when: labels -1 or +1, both present.

```
$ data-science-ml solve ml_hinge_loss "y_true=[1, -1, 1, -1]" "scores=[2, -0.5, 0.3, 0.8]"
hinge = 0.75
squared_hinge = 0.995
n_violations = 3
  by: Hinge and squared hinge loss  (exact fractions)
  assumes: labels -1 or +1, both present
```

What the worked example means: This calculation determines the average penalty for misclassification and low confidence in a binary classifier using labels of negative one or positive one. By comparing true labels with decision scores, it computes the mean hinge loss as 0.75 and the mean squared hinge loss as 0.995. The result indicates that 3 samples failed to meet the required margin threshold, suggesting how much adjustment is needed to improve separation.

Huber, smooth L1 and pseudo-Huber loss — `ml_huber_loss`

The mean Huber loss with threshold delta (quadratic for small errors, linear for large), the smooth L1 loss (Huber / delta) and the pseudo-Huber loss.

$$L_\delta(r) = \begin{cases} \frac{1}{2}r^2 & |r| \le \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right) & |r| > \delta \end{cases}$$

symbol		meaning	SI unit
`y_true`	in	true values	—
`y_pred`	in	predicted values	—
`delta`	in	threshold between the quadratic and linear parts (delta > 0)	—
`huber`	out	mean Huber loss	—
`smooth_l1`	out	mean smooth L1 loss with beta = delta	—
`pseudo_huber`	out	mean pseudo-Huber loss	—

Applies when: delta > 0.

```
$ data-science-ml solve ml_huber_loss "y_true=[3, -0.5, 2, 7]" "y_pred=[2.5, 0, 2, 8]" delta=1
huber = 0.1875
smooth_l1 = 0.1875
pseudo_huber = 0.162570385
  by: Huber, smooth L1 and pseudo-Huber loss  (exact piecewise Huber terms; pseudo-Huber in the cancellation-free form delta^2 x^2
/ (sqrt(1 + x^2) + 1))
  delta = 1
  assumes: delta > 0
```

What the worked example means: This calculation determines three robust loss metrics based on the true values [3, -0.5, 2, 7] and predicted values [2.5, 0, 2, 8] using a threshold of 1. The resulting mean Huber loss is 0.1875, which applies quadratic penalty for small errors and linear for large ones. The smooth L1 loss, calculated as 0.1875, scales this by the threshold, while the pseudo-Huber loss yields 0.16257, offering a differentiable approximation to absolute error.

Log-softmax — `ml_log_softmax`

The log-softmax of logits, z_i - log sum exp(z), computed stably even for very large logits.

$$\log \sigma(z)_i = z_i - m - \log \sum_j e^{z_j - m}$$

symbol		meaning	SI unit
`z`	in	logits	—
`log_probabilities`	out	z_i - logsumexp(z)	—
`log_sum_exp`	out	logsumexp(z)	—

Applies when: at least two logits.

```
$ data-science-ml solve ml_log_softmax "z=[1, 2, 3]"
log_probabilities = [-2.40760596444,-1.40760596444,-0.407605964444]
log_sum_exp = 3.407605964
  by: Log-softmax  (log-sum-exp with the maximum subtracted)
  assumes: at least two logits
```

What the worked example means: This calculation determines the natural logarithm of the softmax probability for a specific input logit. By using the log-sum-exp trick with the maximum subtracted, it ensures numerical stability even when logits are large or small. The result, [-2.40761, -1.40761, -0.407606], represents the confidence score in logarithmic form, while 3.40761 serves as the normalization term that accounts for all competing classes.

Multi-class hinge loss (Crammer-Singer and Weston-Watkins) — `ml_multiclass_hinge_loss`

The multi-class hinge losses of a decision matrix: Crammer-Singer max(0, 1 - f_y + max_{j != y} f_j) and Weston-Watkins sum_{j != y} max(0, 1 - f_y + f_j) / K, averaged over samples.

$$\ell_{CS} = \max\left(0,\, 1 - f_y + \max_{j \neq y} f_j\right), \quad \ell_{WW} = \frac{1}{K}\sum_{j \neq y} \max(0,\, 1 - f_y + f_j)$$

symbol		meaning	SI unit
`y_true`	in	true class index of each sample (0 to K-1)	—
`D`	in	decision values: one row per sample, one column per class (K >= 3)	—
`crammer_singer`	out	mean Crammer-Singer hinge loss	—
`weston_watkins`	out	mean Weston-Watkins hinge loss (sum over wrong classes divided by K)	—

Applies when: K >= 3 classes; labels are column indices.

```
$ data-science-ml solve ml_multiclass_hinge_loss "y_true=[0, 1, 2]" "D=[[1.2, 0.3, -0.5], [0.4, 0.9, 1.1], [-0.2, 0.1, 2]]"
crammer_singer = 0.4333333333
weston_watkins = 0.2
  by: Multi-class hinge loss (Crammer-Singer and Weston-Watkins)  (exact fractions)
  assumes: K >= 3 classes; labels are column indices
```

What the worked example means: This calculation determines two multi-class hinge losses from a matrix of decision values. The Crammer-Singer loss focuses on the most competitive incorrect class, while the Weston-Watkins loss averages the margins against all incorrect classes. For the given inputs, the resulting mean Crammer-Singer loss is 0.433333 and the mean Weston-Watkins loss is 0.2, indicating how well the true class is separated from the others in the support vector machine formulation.

Quantile (pinball) loss — `ml_quantile_loss`

The mean pinball loss of predictions of the alpha-quantile: under-predictions cost alpha per unit, over-predictions 1 - alpha.

$$L_\alpha(y, q) = \max\{\alpha(y - q),\ (\alpha - 1)(y - q)\}$$

symbol		meaning	SI unit
`y_true`	in	true values	—
`y_pred`	in	predicted quantiles	—
`alpha`	in	quantile level (0 < alpha < 1)	—
`pinball`	out	mean pinball loss	—

Applies when: 0 < alpha < 1.

```
$ data-science-ml solve ml_quantile_loss "y_true=[1, 2, 3]" "y_pred=[0, 2, 3]" alpha=0.1
pinball = 0.03333333333
  by: Quantile (pinball) loss  (exact mean of max(alpha (y - q), (alpha - 1)(y - q)))
  alpha = 0.1
  assumes: 0 < alpha < 1
```

What the worked example means: This calculation determines the mean pinball loss for a set of predictions targeting a specific quantile. It uses the true values [1, 2, 3], the predicted quantiles [0, 2, 3], and the quantile level 0.1 to compute the result 0.0333333. The method applies an asymmetric penalty where under-predictions cost alpha per unit error while over-predictions cost one minus alpha, ensuring the model is evaluated specifically on its accuracy for that percentile rather than the average.

Sigmoid (logistic function) and its derivative — `ml_sigmoid`

The logistic sigmoid 1 / (1 + e^-z), its derivative sigma(1 - sigma) and log sigma(z), computed stably for large |z|.

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

symbol		meaning	SI unit
`z`	in	input (logit)	—
`sigmoid`	out	1 / (1 + exp(-z))	—
`derivative`	out	sigma(z)(1 - sigma(z))	—
`log_sigmoid`	out	log sigma(z)	—

Applies when: finite z.

```
$ data-science-ml solve ml_sigmoid z=0
sigmoid = 0.5
derivative = 0.25
log_sigmoid = -0.6931471806
  by: Sigmoid (logistic function) and its derivative  (branch on the sign of z so no exponential overflows; derivative as e^-|z| /
(1 + e^-|z|)^2)
  z = 0
  assumes: finite z
```

What the worked example means: The logistic function converts the input logit 0 into a probability-like value between zero and one, resulting in 0.5. The slope of this curve at that point, which guides optimization steps, is calculated as 0.25. Additionally, the natural logarithm of the output is provided as -0.693147 for numerical stability in loss calculations.

Softmax — `ml_softmax`

The softmax of a vector of logits (numerically stable: the largest logit is subtracted first), with log-sum-exp.

$$\sigma(z)_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}},\ m = \max_j z_j$$

symbol		meaning	SI unit
`z`	in	logits (at least two)	—
`probabilities`	out	softmax probabilities exp(z_i) / sum exp(z_j)	—
`log_sum_exp`	out	log sum exp(z_j)	—

Applies when: at least two logits.

```
$ data-science-ml solve ml_softmax "z=[1, 2, 3]"
probabilities = [0.0900305731704,0.244728471055,0.665240955775]
log_sum_exp = 3.407605964
  by: Softmax  (subtract max(z), exponentiate, normalise (fsum))
  assumes: at least two logits
```

What the worked example means: This calculation converts the raw scores or logits [1, 2, 3] into a normalized probability distribution. The method first identifies the maximum value among the inputs to ensure numerical stability, then exponentiates each shifted value and divides by their sum. The resulting probabilities [0.0900306, 0.244728, 0.665241] indicate the relative likelihood of each class, while the intermediate log-sum-exp value 3.40761 represents the logarithm of the total unnormalized weight, which is essential for calculating cross-entropy loss efficiently without overflow.

Softmax with temperature — `ml_softmax_temperature`

The softmax of logits divided by a temperature T (T > 1 flattens, T < 1 sharpens the distribution) and the entropy of the result in nats.

$$p_i = \frac{e^{z_i / T}}{\sum_j e^{z_j / T}}$$

symbol		meaning	SI unit
`z`	in	logits	—
`T`	in	temperature (T > 0)	—
`probabilities`	out	softmax(z / T)	—
`entropy`	out	entropy of the distribution in nats	—

Applies when: T > 0.

```
$ data-science-ml solve ml_softmax_temperature "z=[1, 2, 3]" T=2
probabilities = [0.186323723226,0.307195885718,0.506480391056]
entropy = 1.020191337
  by: Softmax with temperature  (stable softmax of z / T)
  T = 2
  assumes: T > 0
```

What the worked example means: This calculation applies the softmax function to the logits divided by a temperature parameter. The resulting probabilities are [0.186324, 0.307196, 0.50648], and the entropy of this distribution is 1.02019. A higher temperature value flattens the distribution, increasing uncertainty, while a lower temperature sharpens it, concentrating probability on the highest logit.

Triplet margin loss — `ml_triplet_loss`

The triplet margin loss max(0, d(a, p) - d(a, n) + margin) of an anchor, a positive and a negative embedding, with both Euclidean distances.

$$L = \max\left(0,\ \|a - p\|_2 - \|a - n\|_2 + m\right)$$

symbol		meaning	SI unit
`anchor`	in	anchor embedding	—
`positive`	in	embedding of the same identity	—
`negative`	in	embedding of a different identity	—
`margin`	in	margin (>= 0)	—
`loss`	out	max(0, d_ap - d_an + margin)	—
`d_ap`	out	anchor-positive distance	—
`d_an`	out	anchor-negative distance	—

Applies when: equal lengths; margin >= 0.

```
$ data-science-ml solve ml_triplet_loss "anchor=[0, 0]" "positive=[1, 1]" "negative=[2, 0]" margin=1
loss = 0.4142135624
d_ap = 1.414213562
d_an = 2
  by: Triplet margin loss  (Euclidean distances with fsum)
  margin = 1
  assumes: equal lengths; margin >= 0
```

What the worked example means: This calculation determines the triplet margin loss, a metric used to assess how well an embedding model separates similar items from dissimilar ones. It takes the Euclidean distance between the anchor and positive samples (1.41421) and subtracts the distance between the anchor and negative samples (2), then adds the specified margin (1). The final result (0.414214) represents the penalty applied when the positive sample is not sufficiently closer than the negative sample by at least the margin amount.

Model selection (5)

Bias-variance decomposition of predictions — `ml_bias_variance_decomposition`

For predictions of several models (one row each, e.g. trained on different samples) at the same test points with known noise-free values: the squared bias, the variance and the expected squared error, which equals their sum.

$$E[(\hat{f} - f)^2] = (E[\hat{f}] - f)^2 + E[(\hat{f} - E[\hat{f}])^2]$$

symbol		meaning	SI unit
`P`	in	predictions: one row per trained model, one column per test point (at least two rows)	—
`f`	in	true (noise-free) value at each test point	—
`bias_squared`	out	mean over points of (mean prediction - f)^2	—
`variance`	out	mean over points of the variance of the predictions (divide by the number of models)	—
`expected_mse`	out	mean squared error over models and points = bias^2 + variance	—

Applies when: at least two models.

```
$ data-science-ml solve ml_bias_variance_decomposition "P=[[1, 2], [3, 2.5], [2, 3.5]]" "f=[1.5, 2.5]"
bias_squared = 0.1388888889
variance = 0.5277777778
expected_mse = 0.6666666667
  by: Bias-variance decomposition of predictions  (exact rational sums)
  assumes: at least two models
```

What the worked example means: This calculation decomposes the prediction error into two fundamental sources: bias and variance. Using predictions from multiple models trained on different samples, it computes the mean squared deviation of the average prediction from the true value as the bias-squared term, yielding 0.138889. It also calculates the average spread of individual model predictions around their mean as the variance term, resulting in 0.527778. The sum of these two components equals the total expected mean squared error, reported here as 0.666667, providing insight into whether the model ensemble suffers from underfitting or overfitting.

k-fold cross-validated MSE of linear regression — `ml_kfold_cv_linear_regression`

The mean squared error of ordinary least squares (with intercept) on each of k consecutive, unshuffled folds, trained on the other folds, and the mean over folds.

$$\mathrm{CV}_k = \frac{1}{k}\sum_{i=1}^{k}\mathrm{MSE}_i$$

symbol		meaning	SI unit
`X`	in	features: one row per sample	—
`y`	in	targets	—
`k`	in	number of folds	—
`fold_mse`	out	test MSE of each fold	—
`mean_mse`	out	mean of the fold MSEs	—

Applies when: every training set has at least d + 2 samples and full rank.

```
$ data-science-ml solve ml_kfold_cv_linear_regression "X=[[1], [2], [3], [4], [5], [6]]" "y=[1.1, 1.9, 3.2, 3.9, 5.1, 5.8]" k=3
fold_mse = [0.0925,0.0243295847751,0.01845]
mean_mse = 0.04509319493
  by: k-fold cross-validated MSE of linear regression  (consecutive folds (first n mod k folds one larger); exact least squares by
the normal equations)
  assumes: every training set has at least d + 2 samples and full rank
```

What the worked example means: This calculation determines the generalization performance of a linear regression model by splitting the data into consecutive folds. For each fold, the model is trained on the remaining data using ordinary least squares and tested on the held-out fold to compute its mean squared error. The final result, 0.0450932, is the average of these individual fold errors, such as [0.0925, 0.0243296, 0.01845], providing an estimate of how well the model predicts unseen data under the assumption that training sets have sufficient samples and full rank.

k-fold cross-validation fold sizes — `ml_kfold_sizes`

The test and training set sizes of each fold when n samples are split into k consecutive folds (the first n mod k folds get one extra sample), and the index range of the first test fold.

$$|F_i| = \lfloor n/k \rfloor + \mathbb{1}(i < n \bmod k)$$

symbol		meaning	SI unit
`n`	in	number of samples	—
`k`	in	number of folds	—
`test_sizes`	out	size of each test fold	—
`train_sizes`	out	size of each training set	—
`first_test_fold`	out	first and last index (0-based) of the first test fold	—

Applies when: 2 <= k <= n.

```
$ data-science-ml solve ml_kfold_sizes n=10 k=3
test_sizes = [4,3,3]
train_sizes = [6,7,7]
first_test_fold = [0,3]
  by: k-fold cross-validation fold sizes  (n = k q + r: r folds of q + 1, k - r folds of q, in order)
  assumes: 2 <= k <= n
```

What the worked example means: This calculation determines the precise sizes of test and training sets for each fold in k-fold cross-validation. Given 10 total samples divided into 3 folds, the method distributes any remainder samples to the initial folds. Consequently, the resulting test fold sizes are listed as [4, 3, 3], while the corresponding training set sizes are [6, 7, 7]. The index range for the very first test fold is identified as [0, 3], ensuring that data partitioning follows the standard consecutive block structure.

Leave-one-out cross-validation of linear regression (PRESS) — `ml_loocv_linear_regression`

The leave-one-out cross-validated MSE and the PRESS statistic of ordinary least squares with intercept, from one fit through the leverage shortcut e_i / (1 - h_ii).

$$\mathrm{PRESS} = \sum_i \left(\frac{e_i}{1 - h_{ii}} \right)^2, \quad h_{ii} = x_i^\top (X^\top X)^{-1} x_i$$

symbol		meaning	SI unit
`X`	in	features: one row per sample	—
`y`	in	targets	—
`press`	out	sum of squared leave-one-out errors	—
`loocv_mse`	out	PRESS / n	—

Applies when: at least d + 3 samples; no leverage equal to 1.

```
$ data-science-ml solve ml_loocv_linear_regression "X=[[1], [2], [3], [4], [5], [6]]" "y=[1.1, 1.9, 3.2, 3.9, 5.1, 5.8]"
press = 0.210910889
loocv_mse = 0.03515181483
  by: Leave-one-out cross-validation of linear regression (PRESS)  (hat-matrix diagonal from the exact inverse of the normal
matrix; e_i / (1 - h_ii))
  assumes: at least d + 3 samples; no leverage equal to 1
```

What the worked example means: This calculation determines the predictive performance of a linear regression model using leave-one-out cross validation. It computes the sum of squared errors for each observation when that specific point is excluded from training, known as the PRESS statistic which equals 0.210911. Dividing this total by the number of samples yields the mean squared error estimate 0.0351518, providing an unbiased measure of how well the model generalizes to new data without requiring multiple separate model fits.

Train/test split sizes — `ml_train_test_split_sizes`

The number of test and training samples when n samples are split with a given test fraction (the test size is rounded up).

$$n_{\text{test}} = \lceil t\,n \rceil, \quad n_{\text{train}} = n - n_{\text{test}}$$

symbol		meaning	SI unit
`n`	in	number of samples	—
`test_size`	in	fraction of the samples used for testing (0 to 1, or a percentage)	—
`n_test`	out	ceil(test_size * n)	—
`n_train`	out	n - n_test	—

Applies when: 0 < test_size < 1; a non-empty training set.

```
$ data-science-ml solve ml_train_test_split_sizes n=100 test_size=0.2
n_test = 20
n_train = 80
  by: Train/test split sizes  (exact ceiling of the written fraction times n)
  test_size = 0.2
  assumes: 0 < test_size < 1; a non-empty training set
```

What the worked example means: This calculation determines the specific number of observations assigned to the testing and training subsets. Starting with a total dataset size of 100, the method applies the given test proportion of 0.2. The resulting test set size is calculated by taking the ceiling of the product, which gives 20 samples. The remaining observations form the training set, totaling 80 samples. This approach ensures that the evaluation data is sufficient for statistical validity while maintaining a non-empty training set as required by the conditions.

Neural networks (13)

2-D convolution of an image with a kernel (deep-learning convention) — `ml_conv2d_forward`

The output feature map of a single-channel 2-D convolution as computed in deep learning (cross-correlation: the kernel is not flipped), with a stride and zero padding.

$$Y_{ij} = \sum_{a,b} X_{is+a,\,js+b}\,K_{ab}$$

symbol		meaning	SI unit
`image`	in	input image (rows of pixel values)	—
`kernel`	in	kernel (filter) weights	—
`stride`	in	stride	—
`padding`	in	zero padding on each side	—
`output`	out	output feature map	—
`output_sum`	out	sum of all outputs	—

Applies when: the kernel fits in the padded image.

```
$ data-science-ml solve ml_conv2d_forward "image=[[1, 2, 3, 0], [0, 1, 2, 3], [3, 0, 1, 2], [2, 3, 0, 1]]" "kernel=[[2, 0, 1], [0,
1, 2], [1, 0, 2]]" stride=1 padding=0
output = [[15.0,16.0],[6.0,15.0]]
output_sum = 52
  by: 2-D convolution of an image with a kernel (deep-learning convention)  (exact sums of products over each window)
  assumes: the kernel fits in the padded image
```

What the worked example means: This calculation produces the output feature map [[15, 16], [6, 15]] by applying the kernel [2 0 1; 0 1 2; 1 0 2] to the image [1 2 3 0; 0 1 2 3; 3 0 1 2; 2 3 0 1] using cross-correlation. The process involves sliding the kernel across the padded image with a stride of 1, multiplying overlapping values, and summing them to generate each pixel in the result. The final aggregated value of all pixels in the output map is 52, representing the overall activation level for this specific filter.

Backpropagation gradients of a one-hidden-layer network — `ml_backpropagation`

One backpropagation pass of a dense network with one hidden layer: the loss (mean squared error with identity outputs, or cross-entropy of softmax outputs against a target distribution) and its gradients with respect to every weight and bias.

$$\delta_2 = \frac{\partial L}{\partial z_2},\ \nabla_{W_2} = \delta_2 h^{\top},\ \delta_1 = (W_2^{\top}\delta_2) \odot \phi'(z_1),\ \nabla_{W_1} = \delta_1 x^{\top}$$

symbol		meaning	SI unit
`x`	in	input vector	—
`t`	in	target: values for mse, a probability distribution for cross-entropy	—
`W1`	in	hidden-layer weights (rows = hidden units, columns = inputs)	—
`b1`	in	hidden-layer biases	—
`activation`	in	hidden activation: relu, tanh or sigmoid	—

symbol		meaning	SI unit
`W2`	in	output-layer weights (rows = outputs, columns = hidden units)	—
`b2`	in	output biases	—
`loss`	in	loss: mse (identity outputs, mean over outputs) or cross_entropy (softmax outputs)	—
`loss`	out	loss value	—
`dW1`	out	dL/dW1	—
`db1`	out	dL/db1	—
`dW2`	out	dL/dW2	—
`db2`	out	dL/db2	—

Applies when: shapes agree; no ReLU pre-activation exactly 0; cross-entropy: target sums to 1, at least two outputs.

```
$ data-science-ml solve ml_backpropagation "x=1, 2" t=0.5 "W1=0.5 -0.25; 0.1 0.3" "b1=0.1, -0.2" activation=relu "W2=1 -1" b2=0.05
loss=mse
loss = 0.7225
dW1 = [[-1.7,-3.4],[1.7,3.4]]
db1 = [-1.7,1.7]
dW2 = [[-0.17,-0.85]]
db2 = [-1.7]
  by: Backpropagation gradients of a one-hidden-layer network  (chain rule backwards through the output layer, the activation and
the hidden layer)
  loss = mse
  activation = relu
  assumes: shapes agree; no ReLU pre-activation exactly 0; cross-entropy: target sums to 1, at least two outputs
```

What the worked example means: This calculation determines the sensitivity of the model's error to its parameters. Starting with the output difference, the method applies the chain rule backward through the network layers. The resulting value 0.7225 represents the current prediction error. The gradients [[-0.17, -0.85]], [-1.7], [[-1.7, -3.4], [1.7, 3.4]], and [-1.7, 1.7] indicate how each weight and bias should be adjusted to minimize this error.

Batch normalisation forward pass (training mode) — `ml_batch_norm_forward`

Batch normalisation of a mini-batch in training mode: each feature is normalised with the batch mean and the biased batch variance plus eps, then scaled by gamma and shifted by beta.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta$$

symbol		meaning	SI unit
`X`	in	mini-batch: one row per sample, one column per feature (at least two rows)	—
`gamma`	in	scale per feature	—
`beta`	in	shift per feature	—
`eps`	in	small constant added to the variance (e.g. 1e-5)	—
`output`	out	gamma (x - mean)/sqrt(var + eps) + beta	—
`batch_mean`	out	batch mean of each feature	—
`batch_var`	out	biased batch variance of each feature	—

Applies when: at least two samples; var + eps > 0.

```
$ data-science-ml solve ml_batch_norm_forward "X=[[1, 2], [3, 6], [5, 10]]" "gamma=[1, 1]" "beta=[0, 0]" eps=0
output = [[-1.22474487139,-1.22474487139],[0.0,0.0],[1.22474487139,1.22474487139]]
batch_mean = [3.0,6.0]
batch_var = [2.66666666667,10.6666666667]
  by: Batch normalisation forward pass (training mode)  (exact batch mean and variance; one square root per feature)
  eps = 0
  assumes: at least two samples; var + eps > 0
```

What the worked example means: This calculation performs the forward pass of batch normalization in training mode. It computes the mean [3, 6] and biased variance [2.66667, 10.6667] for each feature across the mini-batch inputs [1 2; 3 6; 5 10]. These statistics are used to standardize the data by subtracting the mean and dividing by the square root of the variance plus epsilon 0. Finally, the standardized values are scaled by gamma [1, 1] and shifted by beta [0, 0] to produce the output [[-1.22474, -1.22474], [0, 0], [1.22474, 1.22474]], which stabilizes the distribution of layer inputs during training.

Convolution output size — `ml_conv_output_size`

The output length (per spatial dimension) of a convolution or pooling window from the input size, kernel size, stride, zero padding and dilation.

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1$$

symbol		meaning	SI unit
`n`	in	input size	—
`kernel`	in	kernel size	—
`stride`	in	stride	—
`padding`	in	zero padding on each side	—
`dilation`	in	dilation (1 = ordinary)	—
`output_size`	out	floor((n + 2p - d(k - 1) - 1) / s) + 1	—
`effective_kernel`	out	d(k - 1) + 1	—

Applies when: the dilated kernel fits in the padded input.

```
$ data-science-ml solve ml_conv_output_size n=32 kernel=3 stride=1 padding=1 dilation=1
output_size = 32
effective_kernel = 3
  by: Convolution output size  (integer floor division)
  assumes: the dilated kernel fits in the padded input
```

What the worked example means: This calculation determines the spatial dimensions of the output feature map produced by a convolution or pooling operation. It uses integer floor division to compute the result from the input size, kernel size, stride, zero padding, and dilation rate. The resulting output size of 32 indicates how many positions the filter can occupy across the padded input, which is essential for designing neural network architectures to ensure tensor shapes match across layers.

Forward pass of a one-hidden-layer dense network — `ml_dense_forward_pass`

The forward pass of a dense network x -> activation(W1 x + b1) -> W2 h + b2 with an identity, sigmoid or softmax output: the hidden activations and the outputs.

$$h = \phi(W_1 x + b_1), \quad \hat{y} = g(W_2 h + b_2)$$

symbol		meaning	SI unit
`x`	in	input vector	—
`W1`	in	hidden-layer weights (rows = hidden units, columns = inputs)	—
`b1`	in	hidden-layer biases	—
`activation`	in	hidden activation: relu, tanh or sigmoid	—
`W2`	in	output-layer weights (rows = outputs, columns = hidden units)	—
`b2`	in	output biases	—
`output`	in	output activation: identity, sigmoid or softmax	—
`hidden`	out	hidden activations h	—
`output`	out	network outputs	—
`max_output`	out	largest output	—

Applies when: shapes agree (W1: hidden x inputs, W2: outputs x hidden).

```
$ data-science-ml solve ml_dense_forward_pass "x=1, 2" "W1=0.5 -0.25; 0.1 0.3" "b1=0.1, -0.2" activation=relu "W2=1 -1" b2=0.05
output=identity
hidden = [0.1,0.5]
output = [-0.35]
max_output = -0.35
  by: Forward pass of a one-hidden-layer dense network  (matrix-vector products with fsum, then the activations)
  activation = relu
  output = identity
  assumes: shapes agree (W1: hidden x inputs, W2: outputs x hidden)
```

What the worked example means: This calculation determines the response of a dense neural network by propagating the input vector [1, 2] through the system. First, the input is transformed by the hidden layer weights [0.5 -0.25; 0.1 0.3] and biases [0.1, -0.2], then passed through the activation function relu to yield the hidden state [0.1, 0.5]. This hidden state is subsequently multiplied by the output weights [1 -1] and added to the output biases [0.05], finally processed by the output activation identity to generate the predictions [-0.35]. The result represents the model's inference for the given input, summarizing both internal representations and final decisions.

Max pooling and average pooling — `ml_pooling`

Max pooling and average pooling of a 2-D feature map with a square window, a stride and no padding.

$$Y_{ij} = \max_{a,b<k} X_{is+a,\, js+b}$$

symbol		meaning	SI unit
`X`	in	feature map	—
`size`	in	window size	—

symbol		meaning	SI unit
`stride`	in	stride	—
`max_pool`	out	maximum of each window	—
`avg_pool`	out	mean of each window	—

Applies when: the window fits in the input.

```
$ data-science-ml solve ml_pooling "X=[[1, 3, 2, 4], [5, 6, 1, 2], [7, 2, 9, 0], [1, 8, 3, 4]]" size=2 stride=2
max_pool = [[6.0,4.0],[8.0,9.0]]
avg_pool = [[3.75,2.25],[4.5,4.0]]
  by: Max pooling and average pooling  (exact window maxima and means)
  assumes: the window fits in the input
```

What the worked example means: This calculation applies a square window of size 2 with a stride of 2 to the input feature map [1 3 2 4; 5 6 1 2; 7 2 9 0; 1 8 3 4]. For each position, it computes the highest value within the window for max pooling and the arithmetic mean for average pooling. The resulting outputs are [[6, 4], [8, 9]] for the maxima and [[3.75, 2.25], [4.5, 4]] for the means, effectively reducing the spatial dimensions while preserving key features.

Multi-head attention (identity projections) — `ml_multihead_attention`

Multi-head attention with identity input and output projections: the model dimension is split into equal heads, scaled dot-product attention runs in each head (scale 1/sqrt(head size)) and the head outputs are concatenated.

$$\mathrm{MHA} = [\mathrm{head}_1, \ldots, \mathrm{head}_h], \quad \mathrm{head}_i = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d/h}}\right) V_i$$

symbol		meaning	SI unit
`Q`	in	queries (rows), width = model dimension	—
`K`	in	keys	—
`V`	in	values	—
`heads`	in	number of heads (divides the model dimension)	—
`output`	out	concatenated head outputs	—
`output_sum`	out	sum of all output entries	—

Applies when: equal widths; model dimension divisible by heads.

```
$ data-science-ml solve ml_multihead_attention "Q=1 0 1 0" "K=1 0 0 1; 0 1 1 0" "V=1 2 3 4; 5 6 7 8" heads=2
output = [[2.32095380269,3.32095380269,5.67904619731,6.67904619731]]
output_sum = 18
  by: Multi-head attention (identity projections)  (per head: softmax(Q_h K_h^T / sqrt(d_h)) V_h; concatenate)
  assumes: equal widths; model dimension divisible by heads
```

What the worked example means: Multi-head attention splits the input dimension into equal heads, computes scaled dot-product attention for each head using queries, keys, and values, and then concatenates the results. The final output represents the combined attention features from all heads, with a total element sum of 18, reflecting the aggregate magnitude of the attended information.

Parameter count of a 2-D convolution layer — `ml_conv_parameter_count`

The number of weights, biases and total parameters of a 2-D convolution layer, including grouped and depthwise convolutions.

$$P = C_{\mathrm{out}} \frac{C_{\mathrm{in}}}{g} k_h k_w + C_{\mathrm{out}}$$

symbol		meaning	SI unit
`in_channels`	in	input channels	—
`out_channels`	in	output channels (filters)	—
`kernel_h`	in	kernel height	—
`kernel_w`	in	kernel width	—
`groups`	in	groups (1 = ordinary; in_channels = depthwise)	—
`bias`	in	whether the layer has a bias per filter	—
`weights`	out	C_out (C_in / g) k_h k_w	—
`biases`	out	C_out or 0	—
`total`	out	weights + biases	—

Applies when: channels divisible by groups.

```
$ data-science-ml solve ml_conv_parameter_count in_channels=3 out_channels=64 kernel_h=3 kernel_w=3 groups=1 bias=True
weights = 1728
biases = 64
total = 1792
  by: Parameter count of a 2-D convolution layer  (integer arithmetic)
  assumes: channels divisible by groups
```

What the worked example means: This calculation determines the parameter count for a two-dimensional convolutional layer. It computes the number of weights using the input channels, output channels, kernel dimensions, and groups, then adds the bias count if enabled. The result shows that there are 1728 weight parameters and 64 bias parameters, summing to 1792 total learnable parameters in the layer.

Parameter count of a dense (fully connected) network — `ml_dense_parameter_count`

The number of trainable parameters of each dense layer and of the whole network, from the layer sizes (input size first), with or without biases.

$$P = \sum_{l} (n_{l-1} n_l + n_l)$$

symbol		meaning	SI unit
`sizes`	in	layer sizes from input to output, e.g. [784, 128, 10]	—
`bias`	in	whether every layer has biases (yes or no)	—
`per_layer`	out	parameters of each layer: n_in n_out (+ n_out biases)	—
`total`	out	total parameters	—

Applies when: positive whole-number sizes.

```
$ data-science-ml solve ml_dense_parameter_count "sizes=[784, 128, 10]" bias=True
per_layer = [100480,1290]
total = 101770
  by: Parameter count of a dense (fully connected) network  (n_in * n_out + n_out per layer)
  assumes: positive whole-number sizes
```

What the worked example means: This calculation determines the count of trainable weights and biases in a dense neural network. It processes the sequence of layer dimensions provided as input, applying the standard formula for fully connected layers where each transition contributes weights equal to the product of the adjacent layer sizes, plus optional biases if enabled. The result, 101770, represents the aggregate parameter count across all layers, which serves as a primary indicator of model capacity and memory requirements.

Parameter count of a transformer encoder — `ml_transformer_parameter_count`

The parameters of a standard transformer encoder layer (multi-head self-attention with biases, a two-layer feed-forward block and two layer norms) and of a stack of such layers.

$$P_{\text{layer}} = 4d^2 + 4d + 2d\,d_{ff} + d_{ff} + d + 4d$$

symbol		meaning	SI unit
`d_model`	in	model (embedding) dimension	—
`n_heads`	in	attention heads (divide d_model)	—
`d_ff`	in	feed-forward hidden size	—
`n_layers`	in	number of layers	—
`attention`	out	4 d^2 + 4 d (Q, K, V and output projections with biases)	—
`feed_forward`	out	2 d d_ff + d_ff + d	—
`layer_norms`	out	4 d	—
`per_layer`	out	parameters per layer	—
`total`	out	parameters of the stack	—

Applies when: d_model divisible by n_heads.

```
$ data-science-ml solve ml_transformer_parameter_count d_model=512 n_heads=8 d_ff=2048 n_layers=6
attention = 1050624
feed_forward = 2099712
layer_norms = 2048
per_layer = 3152384
total = 18914304
  by: Parameter count of a transformer encoder  (integer arithmetic)
  assumes: d_model divisible by n_heads
```

What the worked example means: This calculation determines the number of trainable parameters in a transformer encoder stack. It aggregates weights and biases for multi-head self-attention, feed-forward networks, and layer normalization based on the model dimension 512, head count 8, feed-forward size 2048, and layer depth 6. The result, 18914304, represents the full model size excluding embeddings.

Parameter count of an LSTM, GRU or simple RNN — `ml_rnn_parameter_count`

The trainable parameters of each layer and of a stacked recurrent network (LSTM: 4 gates, GRU: 3, simple RNN: 1), with two bias vectors per gate as in PyTorch, optionally bidirectional.

$$P = G\,h\,(n_{\text{in}} + h) + 2Gh$$

symbol		meaning	SI unit
`cell`	in	cell type: lstm, gru or rnn	—
`input_size`	in	input features	—
`hidden_size`	in	hidden units	—
`num_layers`	in	stacked layers	—
`bidirectional`	in	bidirectional (yes or no)	—
`per_layer`	out	parameters of each layer (both directions)	—
`total`	out	total parameters	—

Applies when: positive whole-number sizes.

```
$ data-science-ml solve ml_rnn_parameter_count cell=lstm input_size=100 hidden_size=128 num_layers=1 bidirectional=False
per_layer = [117760]
total = 117760
  by: Parameter count of an LSTM, GRU or simple RNN  (G h (n_in + h) + 2 G h per direction; layers above the first take h (x 2 if
bidirectional) inputs)
  cell = lstm
  assumes: positive whole-number sizes
```

What the worked example means: This calculation determines the number of trainable weights and biases for a recurrent neural network based on its cell type, hidden size, input features, layer depth, and directionality. The result shows that the specific configuration yields 117760 total parameters, where each individual layer accounts for [117760]. This metric helps in assessing model complexity and memory requirements before training begins.

Receptive field of stacked convolutions — `ml_receptive_field`

The receptive field (how many input positions one output depends on, end to end) and the jump (input distance between adjacent outputs) of a stack of convolution or pooling layers with given kernel sizes, strides and dilations.

$$r = 1 + \sum_{l} (k_l - 1)\, d_l \prod_{i<l} s_i$$

symbol		meaning	SI unit
`kernels`	in	kernel size of each layer (whole numbers)	—
`strides`	in	stride of each layer	—
`dilations`	in	dilation of each layer (1 = ordinary)	—
`receptive_field`	out	1 + sum (k_l - 1) d_l prod_{i<l} s_i	—
`jump`	out	product of the strides	—

Applies when: one kernel, stride and dilation per layer; at most 20 layers.

```
$ data-science-ml solve ml_receptive_field "kernels=[3, 3, 3]" "strides=[1, 1, 1]" "dilations=[1, 1, 1]"
receptive_field = 7
jump = 1
  by: Receptive field of stacked convolutions  (layer-by-layer recursion r += (k - 1) d j, j *= s)
  assumes: one kernel, stride and dilation per layer; at most 20 layers
```

What the worked example means: This calculation determines the receptive field size and the jump distance for a stack of convolutional or pooling layers. The receptive field, given as 7, represents the total number of input positions that influence a single output position after passing through all layers defined by their kernel sizes, strides, and dilations. The jump, calculated as 1, indicates the spacing between adjacent output positions relative to the original input space. These values are derived from the provided layer parameters, specifically the kernels, strides, and dilations.

Scaled dot-product attention — `ml_scaled_dot_product_attention`

Scaled dot-product attention softmax(Q K^T / sqrt(d)) V for given query, key and value matrices, optionally with a causal mask (query i sees keys 0..i): the attention weights and the output.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

symbol		meaning	SI unit
`Q`	in	queries: one row per query	—
`K`	in	keys: one row per key	—
`V`	in	values: one row per key	—

symbol		meaning	SI unit
`causal`	in	apply a causal mask (yes or no)	—
`weights`	out	attention weights (rows sum to 1)	—
`output`	out	weights V	—
`output_sum`	out	sum of all output entries	—

Applies when: Q and K have the same width; K and V have the same rows; causal: as many queries as keys.

```
$ data-science-ml solve ml_scaled_dot_product_attention "Q=[[1, 0], [0, 1]]" "K=[[1, 0], [0, 1], [1, 1]]" "V=[[1, 2], [3, 4], [5,
6]]" causal=False
weights = [[0.40111209268,0.19777581464,0.40111209268],[0.19777581464,0.40111209268,0.40111209268]]
output = [[3.0,4.0],[3.40667255608,4.40667255608]]
output_sum = 14.81334511
  by: Scaled dot-product attention  (scores scaled by 1/sqrt(d), stable softmax per row, weighted sum of values)
  assumes: Q and K have the same width; K and V have the same rows; causal: as many queries as keys
```

What the worked example means: This calculation produces the scaled dot-product attention result, where queries and keys are compared to determine relevance. The resulting attention weights, shown as [[0.401112, 0.197776, 0.401112], [0.197776, 0.401112, 0.401112]], indicate how much each value contributes to the final output. By applying a weighted sum of the values using these normalized scores, the mechanism generates the context-aware representation seen in [[3, 4], [3.40667, 4.40667]]. This process ensures that dependencies between sequence elements are captured effectively while maintaining numerical stability through scaling.

Optimizers (6)

Adam optimizer: one update — `ml_adam_step`

One Adam update: the new first and second moment estimates and the parameters after the bias-corrected step w - lr m_hat / (sqrt(v_hat) + eps).

$$\hat{m} = \frac{m_t}{1-\beta_1^t},\ \hat{v} = \frac{v_t}{1-\beta_2^t},\quad w_t = w_{t-1} - \eta\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon}$$

symbol		meaning	SI unit
`w`	in	parameters	—
`grad`	in	gradient	—
`m`	in	first-moment estimate before the step (zeros at the start)	—
`v`	in	second-moment estimate before the step (zeros at the start)	—
`t`	in	step number after this update (1 for the first step)	—
`lr`	in	learning rate	—
`beta1`	in	first-moment decay (e.g. 0.9)	—
`beta2`	in	second-moment decay (e.g. 0.999)	—
`eps`	in	denominator constant (e.g. 1e-8)	—
`m_new`	out	beta1 m + (1 - beta1) g	—
`v_new`	out	beta2 v + (1 - beta2) g^2	—
`w_new`	out	parameters after the step	—

Applies when: t >= 1; 0 <= beta1, beta2 < 1; v >= 0; equal lengths.

```
$ data-science-ml solve ml_adam_step "w=[1, -2]" "grad=[0.5, -1]" "m=[0, 0]" "v=[0, 0]" t=1 lr=0.001 beta1=0.9 beta2=0.999 eps=1e-
08
m_new = [0.05,-0.1]
v_new = [0.00025,0.001]
w_new = [0.99900000002,-1.99900000001]
  by: Adam optimizer: one update  (PyTorch's update rule (eps added after the square root of the bias-corrected v))
  lr = 0.001
  beta1 = 0.9
  beta2 = 0.999
  eps = 1e-08
  assumes: t >= 1; 0 <= beta1, beta2 < 1; v >= 0; equal lengths
```

What the worked example means: This calculation performs one step of the Adam optimizer. It updates the first-moment estimate to [0.05, -0.1] and the second-moment estimate to [0.00025, 0.001] using the current gradient and previous estimates. These values are then bias-corrected and used to adjust the parameters, resulting in the new weight value [0.999, -1.999]. This process adapts the learning rate for each parameter based on recent gradient activity.

AdamW optimizer (decoupled weight decay): one update — `ml_adamw_step`

One AdamW update: the parameters first decay by the factor (1 - lr weight_decay), then take the bias-corrected Adam step.

$$w \leftarrow w(1-\eta\lambda),\quad w_t = w - \eta\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon}$$

symbol		meaning	SI unit
`w`	in	parameters	—
`grad`	in	gradient	—
`m`	in	first-moment estimate before the step (zeros at the start)	—
`v`	in	second-moment estimate before the step (zeros at the start)	—
`t`	in	step number after this update (1 for the first step)	—
`lr`	in	learning rate	—
`beta1`	in	first-moment decay (e.g. 0.9)	—
`beta2`	in	second-moment decay (e.g. 0.999)	—
`eps`	in	denominator constant (e.g. 1e-8)	—
`weight_decay`	in	decoupled weight decay (e.g. 0.01)	—
`m_new`	out	beta1 m + (1 - beta1) g	—
`v_new`	out	beta2 v + (1 - beta2) g^2	—
`w_new`	out	parameters after decay and step	—

Applies when: t >= 1; 0 <= beta1, beta2 < 1; v >= 0; weight_decay >= 0.

```
$ data-science-ml solve ml_adamw_step "w=[1, -2]" "grad=[0.5, -1]" "m=[0, 0]" "v=[0, 0]" t=1 lr=0.001 beta1=0.9 beta2=0.999
eps=1e-08 weight_decay=0.01
m_new = [0.05,-0.1]
v_new = [0.00025,0.001]
w_new = [0.99899000002,-1.99898000001]
  by: AdamW optimizer (decoupled weight decay): one update  (PyTorch's AdamW rule)
  lr = 0.001
  beta1 = 0.9
  beta2 = 0.999
  eps = 1e-08
  weight_decay = 0.01
  assumes: t >= 1; 0 <= beta1, beta2 < 1; v >= 0; weight_decay >= 0
```

What the worked example means: This calculation performs one AdamW update step. The current weights [1, -2] are first scaled by the decay term, then adjusted using the bias-corrected gradient estimate derived from the new moment [0.05, -0.1] and variance [0.00025, 0.001]. The resulting weight value is [0.99899, -1.99898], which incorporates both regularization and adaptive learning rate scaling based on the gradient [0.5, -1].

Cosine-annealing learning-rate schedule — `ml_cosine_lr_schedule`

The learning rate at a given epoch under cosine annealing from lr_max down to eta_min over T_max epochs.

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi t}{T}\right)$$

symbol		meaning	SI unit
`lr_max`	in	initial (maximum) learning rate	—
`eta_min`	in	final (minimum) learning rate	—
`T_max`	in	length of the schedule in epochs (at most 1000)	—
`epoch`	in	epoch number (0 to T_max)	—
`lr`	out	eta_min + (lr_max - eta_min)(1 + cos(pi t / T_max)) / 2	—

Applies when: 0 <= eta_min < lr_max; 0 <= epoch <= T_max <= 1000.

```
$ data-science-ml solve ml_cosine_lr_schedule lr_max=0.1 eta_min=0 T_max=100 epoch=50
lr = 0.05
  by: Cosine-annealing learning-rate schedule  (closed form)
  lr_max = 0.1
  eta_min = 0
  assumes: 0 <= eta_min < lr_max; 0 <= epoch <= T_max <= 1000
```

What the worked example means: This calculation determines the current learning rate using a cosine annealing schedule. Starting from an initial maximum of 0.1 and targeting a minimum of 0, the rate smoothly decreases over a total period of 100 epochs. At the specific step of 50, the resulting value is 0.05, reflecting the position on the decay curve.

Linear warm-up followed by cosine decay — `ml_warmup_cosine_lr_schedule`

The learning rate at a given step of the common transformer schedule: linear warm-up from start_factor lr_max to lr_max over the warm-up steps, then cosine decay to eta_min at the last step.

$$\eta_t = \begin{cases} \eta_{\max}\left(f + (1-f)\frac{t}{W}\right) & t < W \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\frac{\pi(t-W)}{T-W}\right) & t \geq W \end{cases}$$

symbol		meaning	SI unit
`lr_max`	in	peak learning rate	—
`start_factor`	in	fraction of lr_max at step 0 (0 < f <= 1)	—
`warmup`	in	warm-up steps	—
`total`	in	total steps (at most 1000)	—
`eta_min`	in	final learning rate	—
`step`	in	step number (0 to total)	—
`lr`	out	learning rate at the step	—

Applies when: 1 <= warmup < total <= 1000; 0 <= step <= total.

```
$ data-science-ml solve ml_warmup_cosine_lr_schedule lr_max=0.001 start_factor=0.1 warmup=10 total=100 eta_min=0 step=5
lr = 0.00055
  by: Linear warm-up followed by cosine decay  (closed form of each phase)
  lr_max = 0.001
  start_factor = 0.1
  eta_min = 0
  assumes: 1 <= warmup < total <= 1000; 0 <= step <= total
```

What the worked example means: This calculation determines the learning rate for a specific training step using a schedule that combines linear warm-up and cosine decay. During the initial phase, the rate increases linearly from a starting fraction of the peak value up to the maximum learning rate over the defined warm-up steps. After this period, it decreases following a cosine curve towards the minimum learning rate by the final step. The result, 0.00055, represents the precise learning rate applied at step 5, balancing rapid initial convergence with stable later-stage refinement.

SGD with momentum: one update — `ml_sgd_momentum_step`

One step of stochastic gradient descent with momentum in PyTorch's form (optional Nesterov and weight decay): d = g + wd w, buf = mu buf + d, w = w - lr (buf, or d + mu buf with Nesterov).

$$b_t = \mu b_{t-1} + g_t, \quad w_t = w_{t-1} - \eta\, b_t$$

symbol		meaning	SI unit
`w`	in	parameters	—
`grad`	in	gradient	—
`buf`	in	momentum buffer before the step (zeros at the start)	—
`lr`	in	learning rate	—
`momentum`	in	momentum mu (0 <= mu < 1)	—
`nesterov`	in	use Nesterov momentum (yes or no)	—
`weight_decay`	in	L2 weight decay (>= 0)	—
`buf_new`	out	momentum buffer after the step	—
`w_new`	out	parameters after the step	—

Applies when: lr > 0; 0 <= momentum < 1; weight_decay >= 0; equal lengths.

```
$ data-science-ml solve ml_sgd_momentum_step "w=[1, -2]" "grad=[0.5, -1]" "buf=[0.1, 0.2]" lr=0.1 momentum=0.9 nesterov=False
weight_decay=0
buf_new = [0.59,-0.82]
w_new = [0.941,-1.918]
  by: SGD with momentum: one update  (exact rational arithmetic)
  lr = 0.1
  momentum = 0.9
  weight_decay = 0
  assumes: lr > 0; 0 <= momentum < 1; weight_decay >= 0; equal lengths
```

What the worked example means: This calculation performs one step of stochastic gradient descent with momentum. It updates the momentum buffer to [0.59, -0.82] by combining the previous buffer state with the current gradient and weight decay term, then adjusts the parameters to [0.941, -1.918] using this accumulated direction scaled by the learning rate.

Step learning-rate schedule — `ml_step_lr_schedule`

The learning rate at a given epoch under step decay: multiplied by gamma every step_size epochs.

$$\eta_t = \eta_0\, \gamma^{\lfloor t/s \rfloor}$$

symbol		meaning	SI unit
`lr0`	in	initial learning rate	—
`step_size`	in	epochs between decays	—
`gamma`	in	decay factor (0 < gamma <= 1)	—

symbol		meaning	SI unit
`epoch`	in	epoch number (0 = start, at most 1000)	—
`lr`	out	lr0 gamma^floor(epoch / step_size)	—
`decays`	out	number of decays so far	—

Applies when: lr0 > 0; 0 < gamma <= 1; 0 <= epoch <= 1000.

```
$ data-science-ml solve ml_step_lr_schedule lr0=0.1 step_size=30 gamma=0.1 epoch=65
lr = 0.001
decays = 2
  by: Step learning-rate schedule  (closed form)
  lr0 = 0.1
  gamma = 0.1
  assumes: lr0 > 0; 0 < gamma <= 1; 0 <= epoch <= 1000
```

What the worked example means: This calculation determines the current learning rate by applying a step decay strategy. Starting from an initial value of 0.1, the rate is multiplied by the decay factor 0.1 every 30 epochs. For the given epoch 65, this results in 2 total decays and a final learning rate of 0.001. This mechanism helps stabilize convergence during long training cycles by gradually reducing the magnitude of weight updates as training progresses.

Preprocessing (4)

Label encoding and one-hot encoding of categories — `ml_one_hot_encoding`

Encode a list of category values: the categories in sorted order, how often each occurs, the integer code (label encoding) of each value and its one-hot row.

$$\mathbf{onehot}(v)_j = \mathbb{1}(v = c_j)$$

symbol		meaning	SI unit
`categories`	in	category values, written as a bracketed list of words: [red, green, red]	—
`categories`	out	distinct categories, sorted	—
`counts`	out	occurrences of each category	—
`codes`	out	label code of each value (position of its category)	—
`one_hot`	out	one-hot row of each value	—

Applies when: non-empty category words.

```
$ data-science-ml solve ml_one_hot_encoding "categories=red, green, red, blue, green, red"
categories = ["blue","green","red"]
counts = [1,2,3]
codes = [2,1,2,0,1,2]
one_hot = [[0,0,1],[0,1,0],[0,0,1],[1,0,0],[0,1,0],[0,0,1]]
  by: Label encoding and one-hot encoding of categories  (sort the distinct values; index each value)
  categories = red, green, red, blue, green, red
  assumes: non-empty category words
```

What the worked example means: This process takes the input list of category values, red, green, red, blue, green, red, and identifies the unique entries to form a sorted list of distinct categories, [blue, green, red]. It then calculates how many times each category appears in the original data, resulting in [1, 2, 3]. Each value in the input is assigned an integer label code based on its position in the sorted category list, producing [2, 1, 2, 0, 1, 2]. Finally, each value is converted into a binary vector where only the element corresponding to its category index is active, yielding the one-hot representation [[0, 0, 1], [0, 1, 0], [0, 0, 1], [1, 0, 0], [0, 1, 0], [0, 0, 1]]. This encoding allows models to handle categorical data without assuming any numerical order between the categories.

Min-max scaling to [0, 1] — `ml_minmax_scaler`

Scale each feature column linearly to the range [0, 1] using its minimum and maximum (a constant column scales to 0).

$$x' = \frac{x - \mathbf{min}_j}{\mathbf{max}_j - \mathbf{min}_j}$$

symbol		meaning	SI unit
`X`	in	data: one row per sample, one column per feature	—
`mins`	out	column minima	—
`maxs`	out	column maxima	—
`scaled`	out	(x - min) / (max - min)	—

Applies when: a constant column scales to 0.

```
$ data-science-ml solve ml_minmax_scaler "X=[[1, 10], [2, 20], [3, 40], [5, 30]]"
mins = [1.0,10.0]
maxs = [5.0,40.0]
scaled = [[0.0,0.0],[0.25,0.333333333333],[0.5,1.0],[1.0,0.666666666667]]
  by: Min-max scaling to [0, 1]  (exact rational arithmetic)
  assumes: a constant column scales to 0
```

In the worked example, X = [1 10; 2 20; 3 40; 5 30] go through the equation above and give mins = [1, 10], maxs = [5, 40], scaled = [[0, 0], [0.25, 0.3333333333], [0.5, 1], [1, 0.6666666667]]. It applies only when a constant column scales to 0: outside that it says what is wrong instead of returning a number.

Robust scaling (median and interquartile range) — `ml_robust_scaler`

Scale each feature by subtracting its median and dividing by its interquartile range (quartiles by linear interpolation), which resists outliers; a column with zero IQR is only centred.

$$x' = \frac{x - \text{median}}{Q_3 - Q_1}$$

symbol		meaning	SI unit
`X`	in	data: one row per sample, one column per feature	—
`medians`	out	column medians	—
`iqr`	out	column interquartile ranges Q3 - Q1	—
`scaled`	out	(x - median) / IQR	—

Applies when: zero IQR: centred only.

```
$ data-science-ml solve ml_robust_scaler "X=[[1], [2], [3], [4], [100]]"
medians = [3.0]
iqr = [2.0]
scaled = [[-1.0],[-0.5],[0.0],[0.5],[48.5]]
  by: Robust scaling (median and interquartile range)  (quartiles at position (n - 1) p with linear interpolation; exact
fractions)
  assumes: zero IQR: centred only
```

What the worked example means: Robust scaling computes the median and interquartile range for each column in the input data [1; 2; 3; 4; 100]. The transformed value [[-1], [-0.5], [0], [0.5], [48.5]] is obtained by subtracting the median [3] from the original observation and dividing by the interquartile range [2]. This method resists outliers because it relies on quartiles rather than mean and standard deviation. If the interquartile range is zero, the feature is only centered by subtracting the median.

Standardisation (z-scores of features) — `ml_standard_scaler`

Standardise each feature column to mean 0 and unit variance: the column means, the population standard deviations and the scaled matrix (a constant column scales to 0).

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}, \quad s_j^2 = \frac{1}{n}\sum_i (x_{ij} - \bar{x}_j)^2$$

symbol		meaning	SI unit
`X`	in	data: one row per sample, one column per feature	—
`means`	out	column means	—
`stds`	out	population standard deviations (divide by n)	—
`scaled`	out	(x - mean) / std	—

Applies when: a constant column scales to 0.

```
$ data-science-ml solve ml_standard_scaler "X=[[1, 10], [2, 20], [3, 30], [4, 40]]"
means = [2.5,25.0]
stds = [1.11803398875,11.1803398875]
scaled = [[-1.3416407865,-1.3416407865],[-0.4472135955,-0.4472135955],[0.4472135955,0.4472135955],[1.3416407865,1.3416407865]]
  by: Standardisation (z-scores of features)  (exact means and variances; one square root per column)
  assumes: a constant column scales to 0
```

What the worked example means: Standardisation transforms each feature column so that it has a mean of zero and a variance of one. This process calculates the average value for every column, resulting in [2.5, 25], and determines the spread using the population standard deviation formula, yielding [1.11803, 11.1803]. The final output is the scaled matrix [[-1.34164, -1.34164], [-0.447214, -0.447214], [0.447214, 0.447214], [1.34164, 1.34164]], where each original data point from [1 10; 2 20; 3 30; 4 40] is adjusted by subtracting its column mean and dividing by its column standard deviation. If a column contains identical values throughout, its standard deviation is zero, causing the entire scaled column to become zero.

Ranking metrics (4)

Mean average precision (MAP) of rankings — `ml_mean_average_precision`

The average precision of each query's ranked list (mean of precision at the rank of each relevant item, over the relevant items in the list) and their mean, MAP.

$$\mathrm{AP} = \frac{1}{|\mathrm{rel}|} \sum_{k:\,\mathrm{rel}_k=1} P@k, \quad \mathrm{MAP} = \frac{1}{Q} \sum_{q} \mathrm{AP}_q$$

symbol		meaning	SI unit
`R`	in	relevance (0 or 1) in rank order: one row per query, rows of equal length	—
`average_precision`	out	AP of each query	—
`map`	out	mean of the APs	—

Applies when: relevance 0 or 1; every query has a relevant item.

```
$ data-science-ml solve ml_mean_average_precision "R=[[1, 0, 1, 0, 0], [0, 1, 0, 0, 1]]"
average_precision = [0.833333333333,0.45]
map = 0.6416666667
  by: Mean average precision (MAP) of rankings  (exact fractions)
  assumes: relevance 0 or 1; every query has a relevant item
```

What the worked example means: This calculation determines the Mean Average Precision by first computing the Average Precision for each query. The Average Precision is derived from the relevance scores of ranked items, specifically averaging the precision at each rank where a relevant item appears. The final result, 0.641667, represents the mean of these individual query scores, indicating how well the system ranks relevant items near the top of the list across all queries.

Mean reciprocal rank (MRR) — `ml_mean_reciprocal_rank`

The reciprocal rank of the first relevant item of each query's ranked list (0 when none is relevant) and their mean, the MRR.

$$\mathrm{MRR} = \frac{1}{Q} \sum_{q} \frac{1}{\mathrm{rank}_q}$$

symbol		meaning	SI unit
`R`	in	relevance (0 or 1) in rank order: one row per query	—
`reciprocal_ranks`	out	1 / rank of the first relevant item of each query	—
`mrr`	out	mean reciprocal rank	—

Applies when: relevance 0 or 1.

```
$ data-science-ml solve ml_mean_reciprocal_rank "R=[[0, 0, 1], [0, 1, 0], [1, 0, 0]]"
reciprocal_ranks = [0.333333333333,0.5,1.0]
mrr = 0.6111111111
  by: Mean reciprocal rank (MRR)  (exact fractions)
  assumes: relevance 0 or 1
```

What the worked example means: This calculation determines the Mean Reciprocal Rank by averaging the reciprocal ranks for all queries. Each query's score depends on the position of its first relevant item, where relevance is binary. The resulting value, 0.611111, indicates how well the system places correct answers near the top of the list. A higher value signifies better performance in retrieving relevant items early.

NDCG@k (normalised discounted cumulative gain) — `ml_ndcg_at_k`

DCG@k of a ranked list with graded relevance (linear gain rel or exponential gain 2^rel - 1, discount 1/log2(rank + 1)), the ideal DCG@k of the same items sorted by relevance, and NDCG@k = DCG/IDCG.

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)}, \quad \mathrm{NDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}$$

symbol		meaning	SI unit
`rel`	in	relevance grade of each item in rank order (>= 0)	—
`k`	in	cut-off rank	—
`gain`	in	gain: linear (rel) or exponential (2^rel - 1)	—
`dcg`	out	DCG@k	—
`idcg`	out	ideal DCG@k	—
`ndcg`	out	DCG@k / IDCG@k	—

Applies when: grades >= 0; 1 <= k <= list length; at least one relevant item.

```
$ data-science-ml solve ml_ndcg_at_k "rel=[3, 2, 3, 0, 1, 2]" k=6 gain=linear
dcg = 6.861126689
idcg = 7.140995184
ndcg = 0.9608081943
  by: NDCG@k (normalised discounted cumulative gain)  (discounted sums with fsum)
  gain = linear
  assumes: grades >= 0; 1 <= k <= list length; at least one relevant item
```

What the worked example means: This calculation determines the quality of a ranked list by comparing its actual discounted cumulative gain against the ideal possible gain for the same items. The result, 0.960808, is derived from the input relevance grades and the chosen cut-off rank. A score closer to one suggests that highly relevant items are positioned near the top of the list, while a lower score indicates that relevant items are buried deeper in the rankings or that the ordering is suboptimal relative to the perfect arrangement.

Precision@k and recall@k of a ranking — `ml_precision_recall_at_k`

Precision at k and recall at k of a ranked list (relevance 0/1 in rank order), with the number of relevant items in the top k and the hit indicator.

$$P@k = \frac{|\mathrm{rel} \cap \mathrm{top}_k|}{k}, \quad R@k = \frac{|\mathrm{rel} \cap \mathrm{top}_k|}{|\mathrm{rel}|}$$

symbol		meaning	SI unit
`rel`	in	relevance of each ranked item, in rank order (0 or 1)	—
`n_relevant`	in	total number of relevant items for the query	—
`k`	in	cut-off rank	—
`precision_at_k`	out	relevant in top k / k	—
`recall_at_k`	out	relevant in top k / n_relevant	—
`hits`	out	relevant items in the top k	—
`hit_rate`	out	1 if any relevant item is in the top k	—

Applies when: relevance 0 or 1; 1 <= k <= list length; n_relevant >= relevant items in the list.

```
$ data-science-ml solve ml_precision_recall_at_k "rel=[1, 0, 1, 1, 0, 0, 1]" n_relevant=5 k=5
precision_at_k = 0.6
recall_at_k = 0.6
hits = 3
hit_rate = 1
  by: Precision@k and recall@k of a ranking  (exact counts)
  assumes: relevance 0 or 1; 1 <= k <= list length; n_relevant >= relevant items in the list
```

What the worked example means: This calculation determines the quality of a ranked list by measuring how many relevant items appear in the top positions. Using the relevance indicators for each item, the total number of relevant items, and the specified cut-off rank gives the proportion of relevant items within the top k results. The result indicates that 0.6 of the top 5 items are relevant, while 0.6 of all available relevant items were successfully retrieved. This helps assess whether the ranking system effectively surfaces important information early in the list.

Regression metrics (5)

Adjusted R^2 — `ml_adjusted_r2`

R^2 and the adjusted R^2 that penalises the number of predictors p of the model.

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n - 1}{n - p - 1}$$

symbol		meaning	SI unit
`y_true`	in	true values	—
`y_pred`	in	fitted values	—
`p`	in	number of predictors (excluding the intercept)	—
`r2`	out	R^2	—
`adjusted_r2`	out	1 - (1 - R^2)(n - 1)/(n - p - 1)	—

Applies when: n - p - 1 > 0; true values not all equal.

```
$ data-science-ml solve ml_adjusted_r2 "y_true=[10, 20, 30, 40, 50]" "y_pred=[12, 18, 33, 37, 52]" p=1
r2 = 0.97
adjusted_r2 = 0.96
  by: Adjusted R^2  (exact rational arithmetic)
  assumes: n - p - 1 > 0; true values not all equal
```

What the worked example means: This calculation determines the adjusted coefficient of determination, which refines the standard R squared by accounting for the number of predictors in the model. Using the fitted values [12, 18, 33, 37, 52] and true values [10, 20, 30, 40, 50], along with the predictor count 1, it produces an R squared of 0.97 and an adjusted R squared of 0.96. The adjustment penalizes model

complexity, ensuring that additional variables only improve the score if they significantly enhance explanatory power relative to the sample size.

Mean absolute percentage error — `ml_mape`

The mean absolute percentage error of predictions, as a fraction and in per cent.

$$\mathrm{MAPE} = \frac{1}{n}\sum_i \frac{|y_i - \hat{y}_i|}{|y_i|}$$

symbol		meaning	SI unit
`y_true`	in	true values (none equal to 0)	—
`y_pred`	in	predicted values	—
`mape`	out	mean of |y - y_hat| / |y| (a fraction)	—
`mape_percent`	out	the same in per cent	—

Applies when: no true value equal to 0.

```
$ data-science-ml solve ml_mape "y_true=[100, 200, 300]" "y_pred=[110, 190, 330]"
mape = 0.08333333333
mape_percent = 8.333333333
  by: Mean absolute percentage error  (exact mean of the relative absolute errors)
  assumes: no true value equal to 0
```

What the worked example means: This calculation determines the average magnitude of relative errors between true and predicted values. It sums the absolute differences divided by the absolute true values for each data point, then divides by the count. The result, expressed as 0.0833333, indicates the typical prediction error size relative to the actual value scale.

Mean squared logarithmic error — `ml_mean_squared_log_error`

The mean squared logarithmic error ln(1 + y) - ln(1 + y_hat) and its square root (RMSLE), for non-negative targets such as counts and prices.

$$\mathrm{MSLE} = \frac{1}{n}\sum_i \left(\ln(1 + y_i) - \ln(1 + \hat{y}_i)\right)^2$$

symbol		meaning	SI unit
`y_true`	in	true values (>= 0)	—
`y_pred`	in	predicted values (>= 0)	—
`msle`	out	mean squared log error	—
`rmsle`	out	root mean squared log error	—

Applies when: values >= 0.

```
$ data-science-ml solve ml_mean_squared_log_error "y_true=[3, 5, 2.5, 7]" "y_pred=[2.5, 5, 4, 8]"
msle = 0.03973012298
rmsle = 0.1993241656
  by: Mean squared logarithmic error  (log1p differences squared and averaged)
  assumes: values >= 0
```

What the worked example means: This calculation determines the mean squared logarithmic error by taking the natural logarithm of one plus each true value and subtracting the natural logarithm of one plus each predicted value, then squaring these differences and averaging them. The resulting metric, reported as 0.0397301, quantifies the average squared relative difference between predictions and actuals on a logarithmic scale. The square root of this value, given as 0.199324, provides an interpretable measure of typical log-scale deviation, useful for understanding prediction accuracy in terms of multiplicative factors rather than absolute units.

Mean Tweedie deviance (normal, Poisson, gamma, compound Poisson-gamma) — `ml_tweedie_deviance`

The mean Tweedie deviance of predictions for a stated power: 0 squared error, 1 Poisson, 2 gamma, 3 inverse Gaussian, between 1 and 2 compound Poisson-gamma.

$$d_p(y,\mu) = 2\left(\frac{y^{2-p}}{(1-p)(2-p)} - \frac{y\mu^{1-p}}{1-p} + \frac{\mu^{2-p}}{2-p}\right)$$

symbol		meaning	SI unit
`y_true`	in	true values	—
`y_pred`	in	predicted means (positive for power > 0)	—
`power`	in	Tweedie power p (0, or p >= 1)	—
`deviance`	out	mean Tweedie deviance	—

Applies when: power 0 or >= 1; positive predictions (power > 0); y >= 0 (1 <= p < 2) or y > 0 (p >= 2).

```
$ data-science-ml solve ml_tweedie_deviance "y_true=[2, 0, 1, 4]" "y_pred=[0.5, 0.5, 2, 2]" power=1
deviance = 1.426015132
  by: Mean Tweedie deviance (normal, Poisson, gamma, compound Poisson-gamma)  (unit deviance of the Tweedie family for each
sample, averaged)
  power = 1
  assumes: power 0 or >= 1; positive predictions (power > 0); y >= 0 (1 <= p < 2) or y > 0 (p >= 2)
```

What the worked example means: This calculation determines the average unit deviance between true values and predicted means based on the specified Tweedie power. By applying the formula to each sample, it quantifies how well the model fits data with variance structures typical of normal, Poisson, gamma, or compound Poisson-gamma distributions. The resulting value of 1.42602 indicates the magnitude of prediction error relative to the assumed distribution family.

Regression metrics (MAE, MSE, RMSE, R^2, explained variance) — `ml_regression_metrics`

The standard regression error metrics of predictions against true values: mean absolute error, mean squared error, root mean squared error, R^2, explained variance, median absolute error and maximum error.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}, \quad \mathrm{RMSE} = \sqrt{\tfrac{1}{n}\sum (y_i - \hat{y}_i)^2}$$

symbol		meaning	SI unit
`y_true`	in	true values	—
`y_pred`	in	predicted values	—
`mae`	out	mean absolute error	—
`mse`	out	mean squared error	—
`rmse`	out	root mean squared error	—
`r2`	out	coefficient of determination 1 - SS_res / SS_tot	—
`explained_variance`	out	1 - Var(y - y_hat) / Var(y)	—
`median_ae`	out	median absolute error	—
`max_error`	out	largest absolute error	—

Applies when: at least two values; the true values are not all equal.

```
$ data-science-ml solve ml_regression_metrics "y_true=[3, -0.5, 2, 7]" "y_pred=[2.5, 0, 2, 8]"
mae = 0.5
mse = 0.375
rmse = 0.6123724357
r2 = 0.948608137
explained_variance = 0.9571734475
median_ae = 0.5
max_error = 1
  by: Regression metrics (MAE, MSE, RMSE, R^2, explained variance)  (exact rational sums of the errors; the square root rounded
once)
  assumes: at least two values; the true values are not all equal
```

What the worked example means: This calculation evaluates regression performance by comparing predicted values against true values. The mean absolute error, shown as 0.5, represents the average magnitude of errors in the predictions without considering direction. The root mean squared error is 0.612372, which penalizes larger errors more heavily due to squaring. An R-squared value of 0.948608 indicates the proportion of variance in the dependent variable that is predictable from the independent variables. Additionally, the median absolute error is 0.5, providing a robust measure less sensitive to outliers than the mean.